Reference contents

Technical documentation • 2026-10-25

Overview 2
Core API 3
Conversion and arithmetic 7
Angles and trigonometry 10
Temperature, logarithmic and contextual units 13
Catalogue 18
Decimal and binary prefixes 38
Migration from historical units 41
Design and API contract 46
Sources 50
Testing 54
Naming conventions 57
Validation and status 59
Documentation builds 65
Plan y avance 67

Open Graf technical documentation

Open Graf provides reusable components written in Graf. This documentation is organized as an API and engineering reference.

The new unit core is implemented under `src/units`: dimensions, 87 original families, local definitions, immutable measurements, conversion, arithmetic, dimensions, powers, roots, comparisons, `radians`, `degrees` and circular/inverse trigonometry with explicit precision. The catalogue contains 290 factories and the prefix API implements all 24 SI and 10 binary prefixes. Special units add 12 named factories, for 302 in total, plus locally constructed reference/model-dependent quantities. Other library sections still target an older Graf version.

Quickstart

Use current `Graf/develop` with the `graf` CLI on your path. In an OpenGraf checkout, save or run `examples/units/quickstart.graf`:

```
import(units: graf {path: "../../src/units"});

(units.meters(2).plus(units.centimeters(30)).value(),
 units.radians(units.degrees(30)).value(),
 units.radians(2.5).sin())
// => 23/10, symbolic pi/6, approximately 0.598472144103956
```

Run `graf run examples/units/quickstart.graf` from the repository root. Imports are relative to the source file. A numeric call uses that factory's unit; passing a quantity converts it. Read the core API for definitions and extension, or migration for historical code.

Build the documentation

From the repository root:

```
python -m pip install -r docs/requirements.txt
python3 tools/docs_build.py
python3 tools/docs_build.py --check
```

The build reference lists native dependencies, outputs and visual checks.

Unit core API

The core targets current `Graf/develop`. It provides base/engineering families, local unit definitions, integral dimensions and immutable measurements. Conversion and arithmetic, angles and trigonometry, the catalogue, prefixes and special units build on the same protocols. The design reference records the complete object model.

Import and construct

Import the units directory explicitly. Paths are relative to the source file:

```
// Save as examples/units/core.graf, relative to the repository root.
import(units: graf {path: "../../src/units"});

meters$ = units.length.definition("meter", "m", 1);
length$ = meters$(1.over(3));
changed$ = length$.quantity-with-value(2);

(length$.value(), changed$.value(), length$.symbol,
 length$.dimension.length, length$.compatible-with(changed$))
// => 1/3, 2, "m", 1, yes (one response per value)
```

Run `graf run examples/units/core.graf` from the repository root. The executable example uses this exact source. Importing all of `src` is unnecessary for units; the other sections still use an older Graf API.

A bare number is already expressed in the target definition. Construction does not apply `scale`: `inches$(5).value()` is 5, whatever the definition's scale. The numeric constructor accepts `number?` and stores the supplied value without rounding, promotion, conversion to floating point, or numeric decoration. Integer, arbitrary-size integer, float64, rational and symbolic pi payloads are covered by the executable tests. A number supplied by a host uses the same `number?` protocol; the base core adds no restrictions on numeric backends. Specialized domains can restrict the payload: temperature points and logarithmic values require real numbers, and their construction protocols enforce the documented domains.

Definitions and measurements

`unit` is a class. Each family is an heir of `unit`, a definition is an heir of its family, and a numeric call makes an instance of that definition. `unit` and an unfilled family are abstract: they cannot produce a measurement until a complete definition supplies its metadata.

A measurement composes `amount` and `factory`; it inherits metadata and behavior. It is a sealed composite instance. Its slots cannot be assigned, it cannot be extended or instantiated, and wrapping its constructor with `mutable>>` is refused. `.quantity-with-value(number)` makes a new measurement through the original factory. It never constructs an heir of an existing measurement. Numeric construction and replacement use `factory.quantity(number)`, which extensions can override.

Message	Implemented behavior
`family.definition(id, symbol, scale)`	Local definition, not registered globally
`family.definition(id, symbol, scale, prefixable, exact, source)`	Definition with explicit provenance and flags
`factory(number)` / `factory.quantity(number)`	Immutable measurement with the same numeric payload
`factory(quantity)`	Compatible conversion or explicit compound classification

Message	Implemented behavior
`quantity.coherent-value()`	Numeric payload in the coherent unit
`quantity.value()`	Displayed payload; also available as `quantity.amount`
`quantity.factory`	Explicit reference to the definition that constructed it
`quantity.quantity-with-value(number)`	New measurement in that same definition
`quantity.compatible-with(other)`	Same kind identifier and all eight dimension exponents
`unit.family(kind, dimension)`	Local family inheriting the receiver; copies dimension exponents
`dimension(L, M, T, I, Theta, N, J, info)`	Immutable composite of eight integer exponents
`dimension.same-as(other)`	Field-by-field comparison of exponents

`compatible-with` accepts the kind/dimension protocol of a family, definition or measurement. It checks eligibility; it does not convert or compare amounts. Composite equality requires the same definition parent, amount and factory. Equivalent measurements expressed in different definitions are not converted by `equals`; use `.equal-to` for numeric equality after compatible conversion. See conversion and arithmetic.

Definition metadata

Label	Meaning and default
`id`	Nonempty local identifier, supplied to `definition`
`symbol`	Case-sensitive display symbol, supplied to `definition`
`kind`	Semantic family identifier
`dimension`	Integral dimension value
`scale`	Positive real factor relative to the family's coherent unit
`offset`	Zero by default; explicit affine mappings can override it
`linear-arithmetic`	Capability for dimensional products, quotients, powers and roots; independent of reading sums and scalar scaling
`prefixable`	SI-prefix eligibility, default `no`; see prefixes
`exact`	Definition author's exactness claim, default `yes`
`binary-prefixable`	Binary-prefix eligibility; default `no`
`prefix-name` / `prefix-system`	Prefix provenance; defaults empty / `none`
`scale-uncertainty`	Absolute standard uncertainty of the scale in coherent units; default 0
`reference-edition`	Edition/year for measured factors; default empty
`source`	Provenance text, default `"user-defined"`

Use exact rational factors when the definition is exact. The six-argument form can mark a measured factor as inexact and name its source. No measurement uncertainty model or automatic factor verification is implied by these fields. The catalogue provides reviewed definitions and sources; daltons record the CODATA edition and absolute scale uncertainty. Coherent conversion uses Graf's explicit exact-factor messages, preserving exact rational factors.

Nonpositive factors and empty identifiers fail through Graf predicate casts. Non-numeric payloads, invalid flags, nonintegral dimensions and incomplete instances fail through normal Graf pattern/instantiation checks. The core does not depend on testing assertions to report invalid input.

Dimensions and families

The fields, in constructor order, are:

Field	Dimension
`length`	L
`mass`	M
`duration`	T; fixed durations, not calendar periods
`current`	I
`temperature`	Theta
`amount-of-substance`	N
`luminous-intensity`	J
`information`	info; an independent information dimension

Family	Dimension
`ratio`, `angle`, `solid-angle`	All exponents zero; three distinct kinds
`length`	L
`area`	L^2
`volume`	L^3
`mass`	M
`duration`	T
`current`	I
`temperature-point`, `temperature-difference`	Theta; two distinct kinds
`amount-of-substance`	N
`luminous-intensity`	J
`information`	info
`velocity`	L*T^-1
`acceleration`	L*T^-2

Family	Dimension
`frequency`, `radioactivity`	T^-1; two distinct kinds
`force`	LMT^-2
`pressure`	L^-1MT^-2
`energy`, `torque`	L^2MT^-2; two distinct kinds
`power`	L^2MT^-3
`absorbed-dose`, `equivalent-dose`	L^2*T^-2; two distinct kinds

Temperature and dose families establish distinct kinds. Temperature points support reading sums and scalar scaling, and enforce their point/difference rules and absolute-zero boundary. Explicit radiation-weighting models and other contextual conversions are documented in special units. Concrete factories are listed in the catalogue and special-unit reference.

Local extension

A local family needs a kind and a dimension, with no global registry:

```
rate$ = units.unit.family("example-data-rate",
    units.dimension(0, 0, -1, 0, 0, 0, 0, 1));
bits-per-second$ = rate$.definition("example-bit-per-second", "bit/s", 1);
bits-per-second$(64).value()
// => 64
```

Use Graf inheritance to add behavior or refine metadata. An explicit `override>>` replaces inherited metadata so a read does not return both values. A factory must keep its numeric constructor contract and its metadata immutable; custom construction behavior should override the definition's `quantity` method. Mutating a custom ancestor falls outside this immutable-definition protocol.

The files in `src/units/core` supply the shared base classes. Public unit files mount that directory under an internal alias, so importing units directly or through the library root reaches the same base `unit` and `dimension` classes. Concrete catalogue definitions live in `src/units/core/catalogue-definitions`; public files forward to that shared module, so aliases and library-root imports refer to the same factory. No other legacy component is evaluated by the units examples or tests.

Validation

Run `graf test tests/units` from the repository root. The suites cover the 87 original families and 302 named factories, payload preservation, immutable instances, exact factor metadata, local extension, compatibility, provenance and invalid input. See the validation status for the verified counts and remaining work.

The catalogue reference lists all 87 family vectors, including the 62 engineering families added in step 8. The original 25-family table above remains a guide to the initial core.

Conversion and arithmetic

Step 6 adds operations to the unit core. These examples define local factories explicitly. Step 8 provides a reviewed catalogue and decimal/binary prefixes alongside these extension examples.

Executable example

Save this source as `examples/units/operations.graf` and run `graf run examples/units/operations.graf` from the repository root:

```
import(units: graf {path: "../../src/units"});

meters$ = units.length.definition("meter", "m", 1);
centimeters$ = units.length.definition("centimeter", "cm", 1.over(100));
kilometers$ = units.length.definition("kilometer", "km", 1000);
hours$ = units.duration.definition("hour", "h", 3600);
meters-per-second$ = units.velocity.definition("meter-per-second", "m/s", 1);
kilometers-per-hour$ = units.velocity.definition("kilometer-per-hour", "km/h", 5.over(18));
square-meters$ = units.area.definition("square-meter", "m2", 1);

length$ = meters$(2).plus(centimeters$(30));
speed$ = kilometers$(10).over(hours$(2));
side$ = meters$(square-meters$(9).root(2));

(length$.value(), centimeters$(meters$(2)).value(),
 meters-per-second$(speed$).value(), kilometers-per-hour$(speed$).value(),
 side$.value(), meters$(2).equal-to(centimeters$(200)))
```

The six responses are `23/10`, `200`, `25/18`, `5`, `3`, and `yes`. Graf may print fractions as mixed numbers. The example source is tested with the current interpreter.

Conversion

`factory(number)` interprets the number in that definition's unit. `factory(quantity)` converts a compatible measurement, preserving the target factory and leaving the input unchanged. The mapping is:

- To coherent value: `amount * scale + offset`.
- From coherent value: `(coherent - offset) / scale`.

`quantity.coherent-value()` returns a number, not another quantity. Identity mappings skip multiplication by one and addition of zero, retaining the payload and its numeric backend. Exact rational factors remain rational; no automatic rounding or conversion to binary floating point is introduced by the unit layer.

Named measurements must have the same kind identifier and all eight dimension exponents. Equal dimensions alone do not permit energy-to-torque, frequency-to-radioactivity, angle-to-ratio or absorbed-to-equivalent dose conversion. A custom family follows the same rules as a built-in family.

An unclassified compound can be converted explicitly to a named definition with matching dimensions. For example, force times length can be classified as work or torque. After classification, the result is a named measurement and retains that kind; subsequent direct work-to-torque conversion is refused.

Operations

Message	Result
`a.plus(b)` / `a.minus(b)`	Compatible kinds and dimensions; result in `a.factory`
`a.times(number)` / `a.over(number)`	Displayed payload scaled; `a.factory` retained
`a.times(quantity)`	Coherent compound; dimension exponents added
`a.over(quantity)`	Coherent compound; dimension exponents subtracted
`a.pow(integer)`	Coherent compound; exponents multiplied by the power
`a.root(integer)`	Coherent compound; each exponent divided by the degree
`a.root(integer, precision)`	Same dimension rule, explicit numeric root precision
`a.rounded(places)`	Explicit rounding of displayed payload; factory retained
`a.compared-to(b)`	Numeric backend's -1, 0 or 1 after compatible conversion
`a.equal-to(b)`	Numeric equality after compatible conversion
`a.lt(b)` / `a.lte(b)` / `a.gt(b)` / `a.gte(b)`	Numeric ordering after compatible conversion

Addition and subtraction require a quantity on the right; a bare number has no unit. A compound must be classified before adding it to a named measurement. For example, an area compound is not added directly to square meters, even though its dimensions match. Two compounds with the same dimensions can be added directly; they share the compound kind and coherent scale.

Products and quotients do not consult a global registry or infer a physical kind. Each compound owns an immutable local definition with kind `compound`, scale one, offset zero and an empty symbol. `.is-compound()` identifies the classification protocol. A dimensionless quotient remains a quantity until explicitly classified as a ratio or read with `.value()`.

`.pow(0)` follows Graf's numeric behavior and returns a dimensionless compound. Negative integer powers and negative root degrees are allowed when the numeric backend supports them. A root degree cannot be zero, and every resulting dimension exponent must be an integer. A square root of a length therefore fails; a square root of an area can produce a length compound.

Roots delegate to the numeric backend with a default precision of 15 decimal places, or the explicit precision supplied. Exact roots stay exact. Non-exact roots use that precision. Negative even roots, division by zero and unsupported numeric operations keep the backend's Graf failure behavior. This does not add complex arithmetic that the underlying Graf numeric backend lacks.

`equals(a, b)` still compares composite representation. It performs no unit conversion. Use `.equal-to` to compare compatible physical measurements. Numeric equality is not an approximate tolerance test.

Local definitions and factory construction

A family created by `unit.family(kind, dimension)` participates in conversion, arithmetic and comparison without registration. A definition can be extended with Graf inheritance; use `override>>` when replacing metadata so reads remain single-valued. Arithmetic retains the actual left factory, including inherited methods and refinements.

Every numeric constructor, compatible conversion and replacement payload passes through `factory.quantity(number)`. Its default implementation creates a composite instance with `amount` and the explicit `factory` reference. Extensions that validate or specialize construction should override `quantity` while keeping that contract. Operations returning a named quantity use its factory's `quantity`; compound operations use a fresh coherent compound definition.

`coherent-value-of(number)` and `display-value-of(number)` implement the numeric mapping. `is-convertible-from(quantity)` reports conversion eligibility; it includes explicit compound classification. `compatible-with` remains the stricter same-kind and same-dimension check used by addition and comparisons.

Affine values and temperature

Addition/subtraction operates on displayed readings after converting the right operand to the left unit. Scalar multiplication/division operates on the displayed number. These operations also support offset, logarithmic and referenced units. When factories are identical, the right number is used directly, avoiding needless conversion and logarithmic rounding. The factory's `quantity` protocol validates each result and preserves its metadata.

Dimensional quantity products/quotients, powers and roots require both `linear-arithmetic: yes` and offset zero. Temperature points, logarithmic levels and gauge readings set `linear-arithmetic: no`. Temperature differences retain linear dimensional arithmetic.

Special units implement point/difference arithmetic and absolute-zero validation. Point plus point retains the left point factory; point minus point produces a difference. Point plus/minus difference retains the point factory. Reading arithmetic is distinct from a physical model: adding levels does not automatically calculate combined signal power, and adding temperatures does not calculate the equilibrium temperature of a mixture.

Reading arithmetic example

Save as `examples/units/reading-arithmetic.graf` and run `graf run examples/units/reading-arithmetic.graf`:

```
import(units: graf {path: "../../src/units"});

mw$ = units.si-prefixes.milli(units.watts);
dbm$ = units.logarithmic-reference(mw$(1), units.power-decibels);
dbw$ = units.logarithmic-reference(units.watts(1), units.power-decibels);
gauge$ = units.gauge-pressure(units.pascals(100000), units.bars);

(units.celsius(20).plus(units.celsius(10)).value(),
 units.celsius(20).plus(units.fahrenheit(50)).value(),
 units.celsius(20).times(2).value(),
 units.celsius-differences(10).plus(units.fahrenheit(32)).value(),
 units.power-decibels(20).plus(units.power-bels(1)).value(),
 units.ph(2).plus(units.ph(3)).value(),
 gauge$(1).plus(gauge$(1)).value(), dbm$(20).plus(dbw$(0)).value())
// => 30, 30, 40, 50, 30, 5, 2, 50
```

Tests

Run `graf test tests/units`. The operation suites compare exact expected values, factory identities, dimension vectors and expected failures. Value assertions collect the actual response in a tuple, so an empty stream cannot accidentally skip an assertion. The suites include independent conversion anchors, compatible round trips, local family/factory extensions, negative and zero operands, composed dimensions, roots, precision, and distinct kinds with equal dimensions.

Angles and trigonometry

The public `radians` and `degrees` factories provide conversions with symbolic pi, six circular functions and seven inverse functions. These use the shared `angle` family and Graf's `common.math` implementation.

Import and use

```
// Save as examples/units/angles.graf, relative to the repository root.
import(units: graf {path: "../../src/units"});

(units.radians(20).value(),
 units.radians(units.degrees(30)).value(),
 units.degrees(units.radians(pi)).value(),
 units.radians(20).plus(units.degrees(15)).value().round(6),
 units.radians(2.5).sin(),
 units.degrees(30).sin(0),
 units.degrees.asin(1.over(2)).value(),
 units.degrees.atan2(1, -1).value())
// => 20, symbolic pi/6, 180, 20 + 261799/1000000,
//    149618036025989/250000000000000, 1/2, 30, 135
```

Run `graf run examples/units/angles.graf` from the repository root. A numeric argument is in the factory's own unit: `radians(20)` is twenty radians, `degrees(20)` is twenty degrees. A quantity argument requests conversion. There is no implicit interpretation of a bare number as degrees.

Definitions and source files

Public entry	Shared implementation	Symbol	Coherent scale	Prefix eligibility
`src/units/angle.graf`	`src/units/core/angle.graf`	Family	Radians are coherent	Per definition
`src/units/radians.graf`	`src/units/core/radians.graf`	`rad`	1	Yes
`src/units/degrees.graf`	`src/units/core/degrees.graf`	`°`	pi/180	No

Both scales are exact; pi is a symbolic Graf number. Metadata records BIPM's SI Brochure, ninth edition, version 4.01, Tables 4 and 8. See sources. The prefix API implements that eligibility: radians accept SI prefixes and degrees refuse them.

Each definition inherits `angle`, which inherits `unit`. Measurements are immutable instances with their original number and an explicit `factory` reference. All eight dimension exponents are zero, but the kind is `angle`. Ratios and solid angles remain distinct kinds and are not implicitly converted. Aliases and imports from the library root share the same core definitions.

Mixed sums/subtractions and scalar operations retain the left factory. Angles can also use the common conversion and arithmetic API. `equals` compares representation; `.equal-to` compares compatible numeric values. Symbolic pi expressions can have different trees but the same value.

Circular functions

Message on an angular measurement	Result
`.sin(precision)`	Sine
`.cos(precision)`	Cosine
`.tan(precision)`	Tangent
`.cot(precision)`	Cotangent
`.sec(precision)`	Secant
`.csc(precision)`	Cosecant

Every message converts to coherent radians and delegates to the corresponding `common.math` function. The result is a dimensionless Graf number, with no unit wrapper. Precision is optional and defaults to 15 decimal places. It is an integer parameter on each call; no global setting changes.

Exact special values remain exact, including at precision zero: `degrees(30).sin(0)` is 1/2 and `degrees(90).sin()` is the integer 1. Nonexact values are the rational approximations supplied by `common.math`, which can retain more digits than binary64. A request for 30 places is passed through unchanged. Numeric domain and available-precision errors propagate as Graf exceptions; tangent at 90 degrees is undefined.

Payloads are not normalized modulo a turn. `degrees(360000030).sin()` is 1/2 and its stored value remains 360000030. Nonangular quantities have no circular methods. The numeric constructor accepts Graf numbers; circular functions require the real operands supported by `common.math`.

Inverse functions

Invoke inverse functions on a concrete factory, with dimensionless real numbers:

Factory message	Principal range in radians
`.asin(ratio, precision)`	[-pi/2, pi/2]
`.acos(ratio, precision)`	[0, pi]
`.atan(ratio, precision)`	(-pi/2, pi/2)
`.acot(ratio, precision)`	(0, pi)
`.asec(ratio, precision)`	[0, pi], excluding pi/2
`.acsc(ratio, precision)`	[-pi/2, pi/2], excluding 0
`.atan2(y, x, precision)`	[-pi, pi]; negative x axis is pi

The factory converts the radian result of `common.math` to its own unit and constructs a new immutable measurement through `quantity(number)`. `degrees.asin(0.5).value()` is 30; `radians.asin(1).value()` is symbolic pi/2. `atan2` takes y before x and refuses the origin. The inverse domains and principal ranges are those of Graf; asin/acos require [-1, 1], and asec/acsc require an absolute input of at least 1.

The same optional precision defaults to 15 and measures the numeric calculation in radians **before** conversion to the target unit. A nonexact inverse in degrees can therefore have a symbolic payload containing a rational divided by pi. Use `.rounded(places)` to round the displayed unit explicitly; the inverse's precision argument is not a promise of that many displayed degree decimals. An angular measurement is not a dimensionless ratio argument. An abstract family or an already constructed measurement is not an inverse factory.

Local angular definitions

```
turns$ = units.angle.definition("local-turn", "turn", pi.times(2));
turns$(1.over(4)).sin()             // 1
turns$.asin(1).value()              // 1/4
units.degrees(turns$(1.over(4))).value() // 90
```

Definitions inherit the same circular and inverse methods without registration. Overrides of `quantity(number)` remain the construction protocol for inverse results. Source, exactness, prefix flags and offsets have typed defaults in the base unit so overriding metadata does not remove unit/family membership.

Precision boundary

OpenGraf stores constructor payloads and forwards `common.math` results without an extra float conversion. Subsequent arithmetic follows Graf's numeric promotion rules. Graf currently can promote a rational with components beyond Int64 to `float64`, even without a float operand. Thus an arbitrary-precision sine result is rational, but later `.times(1)` can lose digits. This existing engine behavior is not redefined by units; see the precision question recorded in Graf's `Open-Work-Checklist`. Coherent conversion now explicitly uses `exact-product` / `exact-quotient`, which preserve rational factors; exact circular anchors and symbolic angular conversion do not require generic numeric promotion. Complex numeric payload construction remains supported; complex circular-function support depends on Graf and is not added here.

Temperature, logarithmic and contextual units

These rules are implemented in Graf. Public modules are in `src/units`; shared implementations are in `src/units/core`. The finite `catalogue.definition` relation now includes **302 named factories**: the 290 entries in the linear catalogue and these 12 definitions. Context-dependent factories are constructed locally and are not registered globally.

Factory	Quantity or convention	Mapping
`kelvins`	Temperature point	K = value
`celsius`	Temperature point	K = value + 273.15
`fahrenheit`	Temperature point	K = (value + 459.67) × 5/9
`rankines`	Temperature point	K = value × 5/9
`rankine-differences`	Temperature difference	ΔK = value × 5/9
`power-decibels`	Power-ratio level	value = 10 log10(ratio)
`power-bels`	Power-ratio level	value = log10(ratio)
`power-nepers`	Power-ratio level	value = ln(ratio)/2
`amplitude-decibels`	Amplitude-ratio level	value = 20 log10(ratio)
`amplitude-bels`	Amplitude-ratio level	value = 2 log10(ratio)
`amplitude-nepers`	Amplitude-ratio level	value = ln(ratio)
`ph`	Hydrogen-ion acidity	value = −log10(activity)

Temperature factors and offsets are exact. The six logarithmic level factories name the ratio convention explicitly. Their symbols alone do not choose it. The logarithmic formulas are exact definitions; evaluations can be approximate. Sources distinguish definitions from physical models.

Temperature points and differences

A point accepts a real Graf number and rejects a coherent temperature below zero kelvin. Zero is included. The same validation applies to `quantity`, conversion, `quantity-with-value`, arithmetic results and rounding. A negative Celsius or Fahrenheit coordinate can be valid. These factories model ordinary absolute temperature, not population-inversion negative-temperature models.

Expression	Result
`celsius(fahrenheit(32))`	0 °C point
`fahrenheit(212).minus(celsius(0))`	180 Fahrenheit differences
`fahrenheit(32).plus(kelvin-differences(10))`	50 °F point
`kelvins(0).minus(kelvin-differences(1))`	Failure: below absolute zero
`celsius(20).plus(celsius(10))`	30 °C point

Expression	Result
`celsius(20).plus(fahrenheit(50))`	30 °C point
`celsius(20).times(2)`	40 °C point

Point plus point adds displayed readings in the left scale. The right point is converted to that scale first; the result retains the left factory. Thus `fahrenheit(68).plus(celsius(10))` returns 118 °F: 10 °C converts to 50 °F. This is coordinate arithmetic in the chosen display scale. Scalar multiplication and division also operate on the displayed reading and retain its factory. All results pass through the factory's validation, including the absolute-zero boundary and any custom construction rules.

Point minus point returns a difference in the left point's paired difference factory. Point plus/minus difference retains the left point's factory. Difference plus point also returns a point in that point's factory. Comparisons convert points through kelvins. Points and differences do not convert directly to each other, and a compound cannot be classified as a point.

`kelvin-differences`, `celsius-differences` and `fahrenheit-differences` remain in the linear catalogue. Differences support linear arithmetic and eligible prefixes. Absolute points, including kelvins with offset zero, cannot be prefixed. `rankines` pairs with `rankine-differences`.

A custom point can use:

```
local$ = units.temperature-point.point-definition(
    "local-point", "u", 1, 273.15(0), units.celsius-differences, "local specification");
```

The scale must be positive and equal the paired difference's scale. That factory must have temperature-difference kind and dimensions, with zero offset and linear arithmetic. The shorter inherited `definition(id, symbol, scale)` uses offset zero and constructs a matching local difference factory when needed. Special families override base methods: use `all(units.quantity?)?` to check the measurement protocol without requiring the base unit's exact method shape.

Logarithmic ratios and precision

`power-decibels(20)` is a displayed logarithmic value. It is not a power quantity. `.linear-ratio(precision)` returns its positive, dimensionless ratio. `power-decibels.level(100)` constructs the inverse mapping. Ratios must be real and strictly positive; zero does not create an infinite level. Logarithmic payloads are real. pH is not restricted to the interval 0-14.

Conversions within one kind use `factory(quantity)`, or `factory(quantity, precision)`. Power and amplitude conventions are separate kinds. Direct conversion between them is refused. A physical relationship needs an explicit model, even when particular level values happen to agree.

All logarithmic APIs default to **15 decimal places**. They delegate to `common.math.log10`, `ln`, `exp` and Graf's precision-controlled `pow`. Exact powers can remain exact. For other values, precision applies to the underlying logarithm/exponential before the coefficient is applied; it is not a promise of that many final significant digits. Conversion can involve two rounded evaluations. Use explicit tolerance when checking a nonexact round trip. A rounded inverse power that becomes zero is refused, not silently accepted as a valid positive ratio. The ordinary numeric promotion boundary still applies to later generic arithmetic.

Plus/minus combines compatible displayed levels after converting the right level to the left convention. Scalar multiplication/division scales the displayed level. For example, `power-decibels(20).plus(power-bels(1))` returns 30 dB and `ph(2).plus(ph(3))` returns pH 5. Same-factory arithmetic uses the numbers directly, without evaluating their linear ratios. These operations calculate level readings; signal-power addition or chemical mixing uses an explicit physical model on the linear quantities. Dimensional products, quotients, powers and roots of logarithmic quantities remain unavailable. `.rounded(places)` explicitly rounds the displayed level and retains its factory. Comparison within a kind compares level coordinates; for pH, increasing the coordinate decreases activity.

`logarithmic-unit.logarithmic-definition(id, symbol, kind, logarithm, coefficient, source)` constructs a local factory. `logarithm` is `"decimal"` or `"natural"` and the coefficient must be a nonzero real. The `scale` field is not a substitute for this nonlinear mapping. For unreferenced levels, `.coherent-value()` is the ratio.

Referenced logarithmic levels

`logarithmic-reference(referenceQuantity, conventionFactory)` returns an inheritable unit factory that retains both inputs. The reference must be a positive linear quantity; the convention must be a power- or amplitude-level factory. No reference pressure, impedance or reference power is guessed.

```
milliwatts$ = units.si-prefixes.milli(units.watts);
dbm$ = units.logarithmic-reference(milliwatts$(1), units.power-decibels);
dbw$ = units.logarithmic-reference(units.watts(1), units.power-decibels);
level$ = dbm$.level(units.watts(1));       // 30, with its 1 mW reference retained
dbw$(level$).value()                      // 0, referred to 1 W
units.watts(level$.linear-quantity()).value()  // 1 W
```

`dbm$(30)` also constructs a referenced measurement. Numeric `quantity` follows the normal unit construction protocol. `factory(linearQuantity)` and `.level(linearQuantity, precision)` take a physical quantity; the latter exposes precision. `factory(referencedLevel, precision)` changes references and/or the logarithmic convention. A reference and its value are immutable metadata on every result, including rounded results.

Referenced power/amplitude kinds are distinct from unreferenced ratios. Compatibility also checks the reference's physical kind and dimensions. Thus level comparisons account for differing reference values, and a voltage level cannot convert to a power level. `.linear-quantity(precision)` returns a physical quantity in the reference's factory. Ordinary linear factories refuse direct conversion from a logarithmic quantity.

Referenced levels are dimensionless; their reference retains the underlying physical dimensions. Their internal coherent conversion coordinate is the physical SI value, also available as `.physical-value(precision)`. This mapping supports reference changes, comparisons and conversion of the right operand in reading arithmetic. A sum, difference or scalar result retains the left reference and convention. With the definitions above, `dbm$(20).plus(dbw$(0))` returns 50 dBm because 0 dBW converts to 30 dBm. Dimensional products of levels remain unavailable.

`contextual.referenced-level(value, reference, convention, precision)` constructs the same referenced measurement. `contextual.referenced-quantity(level, reference, precision)` checks that a retained reference agrees with the supplied reference. For an unreferenced ratio-level it explicitly applies the supplied reference.

Amplitude-to-power models

`power-law(exponent, source)` is an immutable model for positive ratios: `powerRatio = amplitudeRatio^exponent`. Its inverse uses the reciprocal exponent. The exponent must be positive. Selecting exponent 2 is an explicit assumption; it does not establish equal impedances for an application.

`contextual.power-level(amplitudeLevel, model, targetPowerFactory, precision)` and `contextual.amplitude-level(powerLevel, model, targetAmplitudeFactory, precision)` use `model.power-ratio(ratio, precision)` or `model.amplitude-ratio(ratio, precision)`. Applications can supply their own model box for different impedances or other physical relationships. Invalid/missing responses propagate as Graf failures.

Gauge pressure

`gauge-pressure(referencePressure, displayPressureFactory)` constructs a local factory. Both inputs must have pressure kind and dimensions and support linear arithmetic. The reference is an explicit nonnegative absolute pressure.

A numeric call stores a gauge coordinate; `.coherent-value()` is `gaugeCoordinate × scale + referencePressure`, in pascals. Negative gauge values are allowed while the resulting absolute pressure remains nonnegative. `.absolute-pressure(targetFactory)` returns a linear pressure quantity; its default target is `pascals`. `factory(absolutePressure)` performs the inverse. Conversion between gauge factories changes the reference explicitly and comparison uses absolute pressure. Direct `pascals(gaugeQuantity)` is refused; use `.absolute-pressure()`.

Gauge sums and differences convert the right gauge reading to the left scale and reference before operating. Scalar multiplication/division scales the displayed reading. Results retain the left reference and factory, and still reject negative absolute pressure. Dimensional products and prefixing remain unavailable; supply the desired linear display factory when constructing the gauge factory. No ambient atmospheric pressure is read.

Activity and pH models

`ph.level(activity, precision)` takes dimensionless hydrogen-ion activity, never a dimensionful concentration. The IUPAC definition uses a molality standard. `activity-model(standard, coefficient, source)` implements the explicitly selected model `activity = coefficient × value/standard`. Standard and value must have the same kind and dimensions: molality or amount concentration. They are positive, as is the coefficient. Molality and molarity do not convert implicitly.

`contextual.acidity(value, model, precision)` calls `model.activity(value)`, then constructs `ph`. A concentration-based model is an explicit approximation or application convention, not an automatic implementation of the molality definition. The caller supplies the standard state, coefficient, provenance and any dependence on temperature, composition or density. Custom model boxes can implement those dependencies. This API does not calculate single-ion activity coefficients.

Chemical-equivalent hardness

`equivalent-basis(name, molarMass, positiveIntegerCharge, source)` stores an explicit equivalent basis, with equivalent mass `molarMass/charge`. Molar mass must have the correct kind/dimensions and be positive. No atomic-weight or valence table is selected automatically.

`basis.definition(id, symbol, scale)` creates a hardness unit with that basis. Scale is relative to kg/m³ **as that basis**. A scale of `0.001(0)` corresponds to mg/L. The basis remains on every measurement. Direct conversions or addition across different bases are refused, even with equal dimensions.

`targetBasis.concentration(massConcentration, sourceBasis)` applies:

`target = source × targetEquivalentMass / sourceEquivalentMass`.

`targetBasis.concentration(hardnessQuantity)` reads the retained source basis. The result is expressed in kg/m³ as the target basis and can be converted to a factory created by that basis. Supplying a conflicting source basis is refused. A plain mass concentration does not identify its chemical species. Hardness is nonnegative. Same-basis linear arithmetic retains the factory and basis. These definitions conservatively carry `exact: no`; a rational nominal molar mass is not proof of physical exactness. General uncertainty propagation and a complete speciation model are outside this API.

Radiation weighting

`radiation-weighting(factor, source)` is an explicit positive, single-component weighting model. `contextual.equivalent-dose(absorbedDose, model)` calls `model.radiation-weight(absorbedDose)` and returns `sieverts` with value `absorbed grays × factor`. Kind, dimensions and nonnegative absorbed dose are checked. No direct gray-to-sievert factor is installed in the unit catalogue.

A custom model can inspect the supplied absorbed quantity. The application must provide the radiation type, energy and tissue context as needed. This single component API does not aggregate radiation fields, compute effective dose or select clinical/protection recommendations.

Executable example and tests

Run `graf run examples/units/special.graf` from the repository root. The example below is the same source. The three special suites and the per-factory catalogue checks run with `graf test tests/units`. Assertions collect actual responses so an empty stream cannot pass a value check. Sequence integration checks use the current `count`, `every`, `last` and `single` deferrevals.

```
import(units: graf {path: "../../src/units"});

milliwatts$ = units.si-prefixes.milli(units.watts);
dbm$ = units.logarithmic-reference(milliwatts$(1), units.power-decibels);
dbw$ = units.logarithmic-reference(units.watts(1), units.power-decibels);
level$ = dbm$.level(units.watts(1));
gauge$ = units.gauge-pressure(units.pascals(101325), units.bars);
activity$ = units.activity-model(units.moles-per-kilogram(1), 1, "explicit ideal molality model");
weighting$ = units.radiation-weighting(20, "single-component demonstration model");
// Synthetic molar masses illustrate the equation, not a physical constants table.
calcium$ = units.equivalent-basis("test-Ca", units.grams-per-mole(40), 2, "synthetic example");
carbonate$ = units.equivalent-basis("test-CaCO3", units.grams-per-mole(100), 2, "synthetic example");
hardness$ = carbonate$.definition("mg-per-liter-as-test-CaCO3", "mg/L as test-CaCO3", 0.001(0));

(units.kelvins(units.celsius(0)).value(),
 units.fahrenheit(212).minus(units.celsius(0)).value(),
 units.fahrenheit(32).plus(units.kelvin-differences(10)).value(),
 units.power-decibels(20).linear-ratio(),
 units.ph.level(0.0000001(0)).value(),
 level$.value(), dbw$(level$).value(), units.watts(level$.linear-quantity()).value(),
 gauge$(1).absolute-pressure().value(),
 units.contextual.acidity(units.moles-per-kilogram(0.0000001(0)), activity$).value(),
 units.contextual.equivalent-dose(units.grays(0.001(0)), weighting$).value(),
 hardness$(carbonate$.concentration(units.milligrams-per-liter(40), calcium$)).value())
// => 273.15, 180, 50, 100, 7, 30, 0, 1, 201325, 7, 0.02, 100
```

Unit catalogue

This table contains **290 linear/angular factories**. Each factory is available as `units.<name>` and in `src/units/<name>.graf`. Import `src/units` as shown in the core API. Prefixes also construct local factories without registration. Symbols are metadata, not alternate labels.

Factors are relative to the coherent unit of the named family. All rational factors are exact unless the row is marked **measured**. Symbolic pi remains symbolic. See sources and reviewed conventions for reference IDs. The separate special-unit reference documents temperature points, logarithmic scales and contextual conversions. This linear catalogue includes temperature **differences**. The `catalogue.definition` relation enumerates all 302 named factories, including the 12 special definitions.

Factory	Family	Symbol	Coherent factor	Source
`meters`	`length`	m	1	BIPM-SI-9-v4.01
`kilometers`	`length`	km	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`hectometers`	`length`	hm	100	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`decameters`	`length`	dam	10	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`decimeters`	`length`	dm	1/10	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`centimeters`	`length`	cm	1/100	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`millimeters`	`length`	mm	1/1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`micrometers`	`length`	µm	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`nanometers`	`length`	nm	1/1000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`picometers`	`length`	pm	1/1000000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
`inches`	`length`	in	127/5000	international-yard-pound-1959
`feet`	`length`	ft	381/1250	international-yard-pound-1959
`yards`	`length`	yd	1143/1250	international-yard-pound-1959
`miles`	`length`	mi	201168/125	international-yard-pound-1959

Factory	Family	Symbol	Coherent factor	Source
fathoms	length	fathom	1143/625	international-yard-pound-1959
furlongs	length	fur	25146/125	international-yard-pound-1959
nautical-miles	length	nmi	1852	BIPM-SI-9-v4.01
astronomical-units	length	au	149597870700	BIPM-SI-9-v4.01
angstroms	length	Å	1/10000000000	BIPM-SI-9-v4.01
square-meters	area	m²	1	BIPM-SI-9-v4.01
cubic-meters	volume	m³	1	BIPM-SI-9-v4.01
square-centimeters	area	cm²	1/10000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
cubic-centimeters	volume	cm³	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
square-millimeters	area	mm²	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
cubic-millimeters	volume	mm³	1/1000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
square-kilometers	area	km²	1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
cubic-kilometers	volume	km³	1000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
square-inches	area	in²	16129/25000000	international-yard-pound-1959
cubic-inches	volume	in³	2048383/125000000000	international-yard-pound-1959
square-feet	area	ft²	145161/1562500	international-yard-pound-1959
cubic-feet	volume	ft³	55306341/1953125000	international-yard-pound-1959
square-yards	area	yd²	1306449/1562500	international-yard-pound-1959
cubic-yards	volume	yd³	1493271207/1953125000	international-yard-pound-1959
ares	area	a	100	BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
hectares	area	ha	10000	BIPM-SI-9-v4.01
international-acres	area	acre	316160658/78125	international-yard-pound-1959
barns	area	b	1/100000000000000000000000000000	BIPM-SI-9-v4.01
liters	volume	L	1/1000	BIPM-SI-9-v4.01
milliliters	volume	mL	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
centiliters	volume	cL	1/100000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
deciliters	volume	dL	1/10000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
hectoliters	volume	hL	1/10	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
us-liquid-gallons	volume	gal (US)	473176473/125000000000	NIST-SP811-2008
us-liquid-quarts	volume	qt (US)	473176473/500000000000	NIST-SP811-2008
us-liquid-pints	volume	pt (US)	473176473/1000000000000	NIST-SP811-2008
us-fluid-ounces	volume	fl oz (US)	473176473/16000000000000	NIST-SP811-2008
imperial-gallons	volume	gal (imp)	454609/100000000	NIST-SP811-2008
imperial-quarts	volume	qt (imp)	454609/400000000	NIST-SP811-2008
imperial-pints	volume	pt (imp)	454609/800000000	NIST-SP811-2008
imperial-fluid-ounces	volume	fl oz (imp)	454609/16000000000	NIST-SP811-2008
petroleum-barrels	volume	bbl	9936705933/62500000000	NIST-SP811-2008
grams	mass	g	1/1000	BIPM-SI-9-v4.01
kilograms	mass	kg	1	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
milligrams	mass	mg	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
micrograms	mass	µg	1/1000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
tonnes	mass	t	1000	BIPM-SI-9-v4.01
pounds	mass	lb	45359237/100000000	international-yard-pound-1959
avoirdupois-ounces	mass	oz	45359237/1600000000	international-yard-pound-1959
grains	mass	gr	6479891/100000000000	international-yard-pound-1959
troy-ounces	mass	oz t	19439673/625000000	international-yard-pound-1959
us-short-tons	mass	ton (US)	45359237/50000	international-yard-pound-1959
imperial-long-tons	mass	ton (imp)	317514659/312500	international-yard-pound-1959
slugs	mass	slug	8896443230521/609600000000	international-yard-pound-1959; standard-gravity-9.80665
metric-carats	mass	ct	1/5000	NIST-SP811-2008
technical-mass-units	mass	utm	196133/20000	standard-gravity-9.80665
daltons	mass	Da	41513476723/25000000000000000000000000000000000 **measured**, u=5.2e-37; CODATA 2022	CODATA-2022-atomic-mass-constant
seconds	duration	s	1	BIPM-SI-9-v4.01
milliseconds	duration	ms	1/1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
microseconds	duration	µs	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
nanoseconds	duration	ns	1/1000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
minutes	duration	min	60	BIPM-SI-9-v4.01
hours	duration	h	3600	BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
days	duration	d	86400	BIPM-SI-9-v4.01
weeks	duration	week	604800	fixed-duration-convention
julian-years	duration	a (Julian)	31557600	fixed-duration-convention
radians	angle	rad	1	BIPM-SI-9-v4.01
degrees	angle	°	pi.over(180)	BIPM-SI-9-v4.01
turns	angle	turn	pi.times(2)	BIPM-SI-9-v4.01
gradians	angle	gon	pi.over(200)	BIPM-SI-9-v4.01
arcminutes	angle	′	pi.over(10800)	BIPM-SI-9-v4.01
arcseconds	angle	″	pi.over(648000)	BIPM-SI-9-v4.01
steradians	solid-angle	sr	1	BIPM-SI-9-v4.01
meters-per-second	velocity	m/s	1	BIPM-SI-9-v4.01
kilometers-per-hour	velocity	km/h	5/18	BIPM-SI-9-v4.01
miles-per-hour	velocity	mi/h	1397/3125	NIST-SP811-2008
knots	velocity	kn	463/900	BIPM-SI-9-v4.01
feet-per-second	velocity	ft/s	381/1250	NIST-SP811-2008
meters-per-second-squared	acceleration	m/s²	1	BIPM-SI-9-v4.01
standard-gravities	acceleration	g_n	196133/20000	NIST-SP811-2008
gals	acceleration	Gal	1/100	NIST-SP811-2008
meters-per-second-cubed	jerk	m/s³	1	BIPM-SI-9-v4.01
radians-per-second	angular-velocity	rad/s	1	BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
degrees-per-second	angular-velocity	°/s	pi.over(180)	BIPM-SI-9-v4.01
revolutions-per-minute	angular-velocity	r/min	pi.over(30)	BIPM-SI-9-v4.01
radians-per-second-squared	angular-acceleration	rad/s²	1	BIPM-SI-9-v4.01
hertz	frequency	Hz	1	BIPM-SI-9-v4.01
kilohertz	frequency	kHz	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
megahertz	frequency	MHz	1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
gigahertz	frequency	GHz	1000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
newtons	force	N	1	BIPM-SI-9-v4.01
kilonewtons	force	kN	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
dynes	force	dyn	1/100000	standard-gravity-9.80665; international-yard-pound-1959
kilogram-forces	force	kgf	196133/20000	standard-gravity-9.80665; international-yard-pound-1959
gram-forces	force	gf	196133/20000000	standard-gravity-9.80665; international-yard-pound-1959
tonne-forces	force	tf	196133/20	standard-gravity-9.80665; international-yard-pound-1959
pound-forces	force	lbf	8896443230521/2000000000000	standard-gravity-9.80665; international-yard-pound-1959
ounce-forces	force	ozf	8896443230521/32000000000000	standard-gravity-9.80665; international-yard-pound-1959
kips	force	kip	8896443230521/2000000000	standard-gravity-9.80665; international-yard-pound-1959

Factory	Family	Symbol	Coherent factor	Source
us-short-ton-forces	force	tonf (US)	8896443230521/1000000000	standard-gravity-9.80665; international-yard-pound-1959
pascals	pressure	Pa	1	BIPM-SI-9-v4.01
kilopascals	pressure	kPa	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
megapascals	pressure	MPa	1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
hectopascals	pressure	hPa	100	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
bars	pressure	bar	100000	NIST-SP811-2008; standard-gravity-9.80665
millibars	pressure	mbar	100	NIST-SP811-2008; standard-gravity-9.80665
standard-atmospheres	pressure	atm	101325	NIST-SP811-2008; standard-gravity-9.80665
technical-atmospheres	pressure	at	196133/2	NIST-SP811-2008; standard-gravity-9.80665
torrs	pressure	Torr	20265/152	NIST-SP811-2008; standard-gravity-9.80665
pounds-per-square-inch	pressure	psi	8896443230521/1290320000	NIST-SP811-2008; standard-gravity-9.80665
pounds-per-square-foot	pressure	psf	8896443230521/185806080000	NIST-SP811-2008; standard-gravity-9.80665
kilogram-forces-per-square-centimeter	pressure	kgf/cm²	196133/2	NIST-SP811-2008; standard-gravity-9.80665
joules	energy	J	1	BIPM-SI-9-v4.01
kilojoules	energy	kJ	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
ergs	energy	erg	1/10000000	NIST-SP811-2008
watt-hours	energy	Wh	3600	NIST-SP811-2008
kilowatt-hours	energy	kWh	3600000	NIST-SP811-2008
thermochemical-calories	energy	cal (th)	523/125	NIST-SP811-2008

Factory	Family	Symbol	Coherent factor	Source
international-table-calories	energy	cal (IT)	10467/2500	NIST-SP811-2008
thermochemical-kilocalories	energy	kcal (th)	4184	NIST-SP811-2008
international-table-kilocalories	energy	kcal (IT)	20934/5	NIST-SP811-2008
international-table-btus	energy	Btu (IT)	52752792631/50000000	NIST-SP811-2008
foot-pound-force-energies	energy	ft·lbf	3389544870828501/2500000000000000	NIST-SP811-2008
kilogram-force-meters-energy	energy	kgf·m	196133/20000	NIST-SP811-2008
electronvolts	energy	eV	801088317/500000000000000000000000000000	BIPM-SI-9-v4.01
kiloelectronvolts	energy	keV	801088317/500000000000000000000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
megaelectronvolts	energy	MeV	801088317/500000000000000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
watts	power	W	1	BIPM-SI-9-v4.01
kilowatts	power	kW	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
megawatts	power	MW	1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
mechanical-horsepower	power	hp (mechanical)	37284993579113511/50000000000000	NIST-SP811-2008
metric-horsepower	power	hp (metric)	588399/800	NIST-SP811-2008
electrical-horsepower	power	hp (electric)	746	NIST-SP811-2008
foot-pound-forces-per-second	power	ft·lbf/s	3389544870828501/2500000000000000	NIST-SP811-2008
international-table-btus-per-second	power	Btu (IT)/s	52752792631/50000000	NIST-SP811-2008
newton-meters	torque	N·m	1	BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
pound-force-feet	torque	lbf·ft	3389544870828501/2500000000000000	NIST-SP811-2008
pound-force-inches	torque	lbf·in	1129848290276167/10000000000000000	NIST-SP811-2008
kilogram-force-meters	torque	kgf·m	196133/20000	NIST-SP811-2008
kilogram-meters-per-second	momentum	kg·m/s	1	BIPM-SI-9-v4.01
kilogram-square-meters-per-second	angular-momentum	kg·m²/s	1	BIPM-SI-9-v4.01
kilogram-square-meters	moment-of-inertia	kg·m²	1	BIPM-SI-9-v4.01
meters-to-the-fourth	area-moment-of-inertia	m⁴	1	BIPM-SI-9-v4.01
cubic-meters-section-modulus	section-modulus	m³	1	BIPM-SI-9-v4.01
kilograms-per-cubic-meter	density	kg/m³	1	BIPM-SI-9-v4.01
cubic-meters-per-kilogram	specific-volume	m³/kg	1	BIPM-SI-9-v4.01
kilograms-per-second	mass-flow	kg/s	1	BIPM-SI-9-v4.01
cubic-meters-per-second	volume-flow	m³/s	1	BIPM-SI-9-v4.01
pascal-seconds	dynamic-viscosity	Pa·s	1	BIPM-SI-9-v4.01
square-meters-per-second	kinematic-viscosity	m²/s	1	BIPM-SI-9-v4.01
newtons-per-meter	surface-tension	N/m	1	BIPM-SI-9-v4.01
reciprocal-meters	wavenumber	m⁻¹	1	BIPM-SI-9-v4.01
kelvin-differences	temperature-difference	K	1	BIPM-SI-9-v4.01
joules-per-kelvin	heat-capacity	J/K	1	BIPM-SI-9-v4.01
joules-per-kilogram-kelvin	specific-heat-capacity	J/(kg·K)	1	BIPM-SI-9-v4.01
watts-per-meter-kelvin	thermal-conductivity	W/(m·K)	1	BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
watts-per-square-meter-kelvin	heat-transfer-coefficient	W/(m²·K)	1	BIPM-SI-9-v4.01
reciprocal-kelvins	thermal-expansion	K⁻¹	1	BIPM-SI-9-v4.01
watts-per-square-meter-heat	heat-flux	W/m²	1	BIPM-SI-9-v4.01
joules-per-kilogram	specific-energy	J/kg	1	BIPM-SI-9-v4.01
amperes	current	A	1	BIPM-SI-9-v4.01
coulombs	charge	C	1	BIPM-SI-9-v4.01
volts	potential	V	1	BIPM-SI-9-v4.01
ohms	resistance	Ω	1	BIPM-SI-9-v4.01
siemens	conductance	S	1	BIPM-SI-9-v4.01
farads	capacitance	F	1	BIPM-SI-9-v4.01
henries	inductance	H	1	BIPM-SI-9-v4.01
webers	magnetic-flux	Wb	1	BIPM-SI-9-v4.01
teslas	magnetic-flux-density	T	1	BIPM-SI-9-v4.01
amperes-per-meter	magnetic-field-strength	A/m	1	BIPM-SI-9-v4.01
ohm-meters	resistivity	Ω·m	1	BIPM-SI-9-v4.01
siemens-per-meter	conductivity	S/m	1	BIPM-SI-9-v4.01
volts-per-meter	electric-field-strength	V/m	1	BIPM-SI-9-v4.01
farads-per-meter	permittivity	F/m	1	BIPM-SI-9-v4.01
henries-per-meter	permeability	H/m	1	BIPM-SI-9-v4.01
reciprocal-henries	magnetic-reluctance	H⁻¹	1	BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
henries-permeance	magnetic-permeance	H	1	BIPM-SI-9-v4.01
ampere-turns	magnetomotive-force	A	1	BIPM-SI-9-v4.01
volt-amperes	apparent-power	V·A	1	BIPM-SI-9-v4.01
vars	reactive-power	var	1	BIPM-SI-9-v4.01
coulombs-per-cubic-meter	electric-charge-density	C/m³	1	BIPM-SI-9-v4.01
amperes-per-square-meter	electric-current-density	A/m²	1	BIPM-SI-9-v4.01
candelas	luminous-intensity	cd	1	BIPM-SI-9-v4.01
lumens	luminous-flux	lm	1	BIPM-SI-9-v4.01
lux	illuminance	lx	1	BIPM-SI-9-v4.01
candelas-per-square-meter	luminance	cd/m²	1	BIPM-SI-9-v4.01
lumen-seconds	luminous-energy	lm·s	1	BIPM-SI-9-v4.01
diopters	optical-power	m⁻¹	1	BIPM-SI-9-v4.01
watts-per-square-meter	irradiance	W/m²	1	BIPM-SI-9-v4.01
watts-per-square-meter-steradian	radiance	W/(m²·sr)	1	BIPM-SI-9-v4.01
moles	amount-of-substance	mol	1	BIPM-SI-9-v4.01
kilograms-per-mole	molar-mass	kg/mol	1	BIPM-SI-9-v4.01
moles-per-cubic-meter	amount-concentration	mol/m³	1	BIPM-SI-9-v4.01
kilograms-per-cubic-meter-concentration	mass-concentration	kg/m³	1	BIPM-SI-9-v4.01
moles-per-kilogram	molality	mol/kg	1	BIPM-SI-9-v4.01
katals	catalytic-activity	kat	1	BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
joules-per-mole	molar-energy	J/mol	1	BIPM-SI-9-v4.01
joules-per-mole-kelvin	molar-heat-capacity	J/(mol·K)	1	BIPM-SI-9-v4.01
becquerels	radioactivity	Bq	1	BIPM-SI-9-v4.01
grays	absorbed-dose	Gy	1	BIPM-SI-9-v4.01
sieverts	equivalent-dose	Sv	1	BIPM-SI-9-v4.01
coulombs-per-kilogram	exposure	C/kg	1	BIPM-SI-9-v4.01
bits	information	bit	1	BIPM-SI-9-v4.01
bits-per-second	information-rate	bit/s	1	BIPM-SI-9-v4.01
kilograms-per-meter	mass-per-length	kg/m	1	BIPM-SI-9-v4.01
kilograms-per-square-meter	mass-per-area	kg/m²	1	BIPM-SI-9-v4.01
celsius-differences	temperature-difference	Δ°C	1	BIPM-SI-9-v4.01
fahrenheit-differences	temperature-difference	Δ°F	5/9	NIST-SP811-2008
grams-per-cubic-centimeter	density	g/cm³	1000	NIST-SP811-2008
pounds-per-cubic-foot	density	lb/ft³	28349523125/1769802912	NIST-SP811-2008
pounds-per-cubic-inch	density	lb/in³	56699046250/2048383	NIST-SP811-2008
liters-per-kilogram	specific-volume	L/kg	1/1000	NIST-SP811-2008
liters-per-second	volume-flow	L/s	1/1000	NIST-SP811-2008
liters-per-minute	volume-flow	L/min	1/60000	NIST-SP811-2008
us-gallons-per-minute	volume-flow	gal (US)/min	157725491/2500000000000	NIST-SP811-2008

Factory	Family	Symbol	Coherent factor	Source
poise	dynamic-viscosity	P	1/10	NIST-SP811-2008
centipoise	dynamic-viscosity	cP	1/1000	NIST-SP811-2008
stokes	kinematic-viscosity	St	1/10000	NIST-SP811-2008
centistokes	kinematic-viscosity	cSt	1/1000000	NIST-SP811-2008
grams-per-mole	molar-mass	g/mol	1/1000	NIST-SP811-2008
moles-per-liter	amount-concentration	mol/L	1000	NIST-SP811-2008
milligrams-per-liter	mass-concentration	mg/L	1/1000	NIST-SP811-2008
ampere-hours	charge	A·h	3600	NIST-SP811-2008
gauss	magnetic-flux-density	G	1/10000	NIST-SP811-2008
maxwells	magnetic-flux	Mx	1/100000000	NIST-SP811-2008
oersteds	magnetic-field-strength	Oe	1000.over(pi.times(4))	NIST-SP811-2008
foot-candles	illuminance	fc	1562500/145161	NIST-SP811-2008
foot-lamberts	luminance	fL	1.over(pi.times(0.09290304(0)))	NIST-SP811-2008
curies	radioactivity	Ci	37000000000	NIST-SP811-2008
rutherfords	radioactivity	Rd	1000000	NIST-SP811-2008
rads	absorbed-dose	rad	1/100	NIST-SP811-2008
rems	equivalent-dose	rem	1/100	NIST-SP811-2008
roentgens	exposure	R	129/500000	NIST-SP811-2008
milliamperes	current	mA	1/1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
millivolts	potential	mV	1/1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
kilovolts	potential	kV	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
kilohms	resistance	kΩ	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
megohms	resistance	MΩ	1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
microfarads	capacitance	µF	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
nanofarads	capacitance	nF	1/1000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
picofarads	capacitance	pF	1/1000000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
millihenries	inductance	mH	1/1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
microteslas	magnetic-flux-density	µT	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
millisieverts	equivalent-dose	mSv	1/1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
microsieverts	equivalent-dose	µSv	1/1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
kilobecquerels	radioactivity	kBq	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
bytes	information	B	8	IEC-80000-13-2025
bytes-per-second	information-rate	B/s	8	IEC-80000-13-2025
kilobits	information	kbit	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
megabits	information	Mbit	1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
gigabits	information	Gbit	1000000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
kilobytes	information	kB	8000	IEC-80000-13-2025; BIPM-SI-9-v4.01
megabytes	information	MB	8000000	IEC-80000-13-2025; BIPM-SI-9-v4.01
gigabytes	information	GB	8000000000	IEC-80000-13-2025; BIPM-SI-9-v4.01
terabytes	information	TB	8000000000000	IEC-80000-13-2025; BIPM-SI-9-v4.01

Factory	Family	Symbol	Coherent factor	Source
kilobits-per-second	information-rate	kbit/s	1000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
megabits-per-second	information-rate	Mbit/s	1000000	BIPM-SI-9-v4.01; BIPM-SI-9-v4.01
kibibytes	information	KiB	8192	IEC-80000-13-2025; IEC-80000-13-2025
kibibits	information	Kibit	1024	BIPM-SI-9-v4.01; IEC-80000-13-2025
mebibytes	information	MiB	8388608	IEC-80000-13-2025; IEC-80000-13-2025
mebibits	information	Mibit	1048576	BIPM-SI-9-v4.01; IEC-80000-13-2025
gibibytes	information	GiB	8589934592	IEC-80000-13-2025; IEC-80000-13-2025
gibibits	information	Gibit	1073741824	BIPM-SI-9-v4.01; IEC-80000-13-2025
tebibytes	information	TiB	8796093022208	IEC-80000-13-2025; IEC-80000-13-2025
tebibits	information	Tibit	1099511627776	BIPM-SI-9-v4.01; IEC-80000-13-2025
pebibytes	information	PiB	9007199254740992	IEC-80000-13-2025; IEC-80000-13-2025
pebibits	information	Pibit	1125899906842624	BIPM-SI-9-v4.01; IEC-80000-13-2025
exbibytes	information	EiB	9223372036854775808	IEC-80000-13-2025; IEC-80000-13-2025
exbibits	information	Eibit	1152921504606846976	BIPM-SI-9-v4.01; IEC-80000-13-2025
zebibytes	information	ZiB	9444732965739290427392	IEC-80000-13-2025; IEC-80000-13-2025
zebibits	information	Zibit	11805916207174113034240	BIPM-SI-9-v4.01; IEC-80000-13-2025
yobibytes	information	YiB	9671406556917033397649408	IEC-80000-13-2025; IEC-80000-13-2025
yobibits	information	Yibit	12089258196146291747061760	BIPM-SI-9-v4.01; IEC-80000-13-2025
robibytes	information	RiB	9903520314283042199192993792	IEC-80000-13-2025; IEC-80000-13-2025
robibits	information	Ribit	123794003928538027489912422 4	BIPM-SI-9-v4.01; IEC-80000-13-2025

Factory	Family	Symbol	Coherent factor	Source
quebibytes	information	QiB	1014120480182583521197362564300 8	IEC-80000-13-2025; IEC-80000-13-2025
quebibits	information	Qibit	126765060022822940149670320537 6	BIPM-SI-9-v4.01; IEC-80000-13-2025
ratios	ratio	1	1	BIPM-SI-9-v4.01
percentages	ratio	%	1/100	BIPM-SI-9-v4.01
per-milles	ratio	‰	1/1000	BIPM-SI-9-v4.01
parts-per-million	ratio	ppm	1/1000000	BIPM-SI-9-v4.01

Family dimensions

The eight exponents are ordered as length, mass, duration, current, temperature, amount of substance, luminous intensity and information. Kinds with equal dimensions are intentionally separate; classification is explicit.

Family	Exponents
absorbed-dose	2, 0, -2, 0, 0, 0, 0, 0
acceleration	1, 0, -2, 0, 0, 0, 0, 0
amount-concentration	-3, 0, 0, 0, 0, 1, 0, 0
amount-of-substance	0, 0, 0, 0, 0, 1, 0, 0
angle	0, 0, 0, 0, 0, 0, 0, 0
angular-acceleration	0, 0, -2, 0, 0, 0, 0, 0
angular-momentum	2, 1, -1, 0, 0, 0, 0, 0
angular-velocity	0, 0, -1, 0, 0, 0, 0, 0
apparent-power	2, 1, -3, 0, 0, 0, 0, 0

Family	Exponents
area	2, 0, 0, 0, 0, 0, 0, 0
area-moment-of-inertia	4, 0, 0, 0, 0, 0, 0, 0
capacitance	-2, -1, 4, 2, 0, 0, 0, 0
catalytic-activity	0, 0, -1, 0, 0, 1, 0, 0
charge	0, 0, 1, 1, 0, 0, 0, 0
conductance	-2, -1, 3, 2, 0, 0, 0, 0
conductivity	-3, -1, 3, 2, 0, 0, 0, 0
current	0, 0, 0, 1, 0, 0, 0, 0
density	-3, 1, 0, 0, 0, 0, 0, 0
duration	0, 0, 1, 0, 0, 0, 0, 0
dynamic-viscosity	-1, 1, -1, 0, 0, 0, 0, 0
electric-charge-density	-3, 0, 1, 1, 0, 0, 0, 0
electric-current-density	-2, 0, 0, 1, 0, 0, 0, 0
electric-field-strength	1, 1, -3, -1, 0, 0, 0, 0
energy	2, 1, -2, 0, 0, 0, 0, 0
equivalent-dose	2, 0, -2, 0, 0, 0, 0, 0
exposure	0, -1, 1, 1, 0, 0, 0, 0
force	1, 1, -2, 0, 0, 0, 0, 0
frequency	0, 0, -1, 0, 0, 0, 0, 0
heat-capacity	2, 1, -2, 0, -1, 0, 0, 0

Family	Exponents
heat-flux	0, 1, -3, 0, 0, 0, 0, 0
heat-transfer-coefficient	0, 1, -3, 0, -1, 0, 0, 0
illuminance	-2, 0, 0, 0, 0, 0, 1, 0
inductance	2, 1, -2, -2, 0, 0, 0, 0
information	0, 0, 0, 0, 0, 0, 0, 1
information-rate	0, 0, -1, 0, 0, 0, 0, 1
irradiance	0, 1, -3, 0, 0, 0, 0, 0
jerk	1, 0, -3, 0, 0, 0, 0, 0
kinematic-viscosity	2, 0, -1, 0, 0, 0, 0, 0
length	1, 0, 0, 0, 0, 0, 0, 0
luminance	-2, 0, 0, 0, 0, 0, 1, 0
luminous-energy	0, 0, 1, 0, 0, 0, 1, 0
luminous-flux	0, 0, 0, 0, 0, 0, 1, 0
luminous-intensity	0, 0, 0, 0, 0, 0, 1, 0
magnetic-field-strength	-1, 0, 0, 1, 0, 0, 0, 0
magnetic-flux	2, 1, -2, -1, 0, 0, 0, 0
magnetic-flux-density	0, 1, -2, -1, 0, 0, 0, 0
magnetic-permeance	2, 1, -2, -2, 0, 0, 0, 0
magnetic-reluctance	-2, -1, 2, 2, 0, 0, 0, 0
magnetomotive-force	0, 0, 0, 1, 0, 0, 0, 0

Family	Exponents
mass	0, 1, 0, 0, 0, 0, 0, 0
mass-concentration	-3, 1, 0, 0, 0, 0, 0, 0
mass-flow	0, 1, -1, 0, 0, 0, 0, 0
mass-per-area	-2, 1, 0, 0, 0, 0, 0, 0
mass-per-length	-1, 1, 0, 0, 0, 0, 0, 0
molality	0, -1, 0, 0, 0, 1, 0, 0
molar-energy	2, 1, -2, 0, 0, -1, 0, 0
molar-heat-capacity	2, 1, -2, 0, -1, -1, 0, 0
molar-mass	0, 1, 0, 0, 0, -1, 0, 0
moment-of-inertia	2, 1, 0, 0, 0, 0, 0, 0
momentum	1, 1, -1, 0, 0, 0, 0, 0
optical-power	-1, 0, 0, 0, 0, 0, 0, 0
permeability	1, 1, -2, -2, 0, 0, 0, 0
permittivity	-3, -1, 4, 2, 0, 0, 0, 0
potential	2, 1, -3, -1, 0, 0, 0, 0
power	2, 1, -3, 0, 0, 0, 0, 0
pressure	-1, 1, -2, 0, 0, 0, 0, 0
radiance	0, 1, -3, 0, 0, 0, 0, 0
radioactivity	0, 0, -1, 0, 0, 0, 0, 0
ratio	0, 0, 0, 0, 0, 0, 0, 0

Family	Exponents
reactive-power	2, 1, -3, 0, 0, 0, 0, 0
resistance	2, 1, -3, -2, 0, 0, 0, 0
resistivity	3, 1, -3, -2, 0, 0, 0, 0
section-modulus	3, 0, 0, 0, 0, 0, 0, 0
solid-angle	0, 0, 0, 0, 0, 0, 0, 0
specific-energy	2, 0, -2, 0, 0, 0, 0, 0
specific-heat-capacity	2, 0, -2, 0, -1, 0, 0, 0
specific-volume	3, -1, 0, 0, 0, 0, 0, 0
surface-tension	0, 1, -2, 0, 0, 0, 0, 0
temperature-difference	0, 0, 0, 0, 1, 0, 0, 0
temperature-point	0, 0, 0, 0, 1, 0, 0, 0
thermal-conductivity	1, 1, -3, 0, -1, 0, 0, 0
thermal-expansion	0, 0, 0, 0, -1, 0, 0, 0
torque	2, 1, -2, 0, 0, 0, 0, 0
velocity	1, 0, -1, 0, 0, 0, 0, 0
volume	3, 0, 0, 0, 0, 0, 0, 0
volume-flow	3, 0, -1, 0, 0, 0, 0, 0
wavenumber	-1, 0, 0, 0, 0, 0, 0, 0

Decimal and binary prefixes

`si-prefixes` contains the 24 BIPM prefixes from quetta to quecto. `binary-prefixes` contains the 10 prefixes from kibi to quebi listed in BIPM version 4.01 and IEC 80000-13:2025. SI prefixes always mean powers of ten; binary prefixes mean powers of two. See reviewed sources.

Use

```
// Save as examples/units/catalogue.graf, relative to the repository root.
import(units: graf {path: "../../src/units"});

quectograms$ = units.si-prefixes.quecto(units.grams);
quebibytes$ = units.binary-prefixes.quebi(units.bytes);

(units.meters(units.feet(3)).value(),
 units.liters(units.us-liquid-gallons(1)).value(),
 units.liters(units.imperial-gallons(1)).value(),
 units.bytes(units.kilobytes(1)).value(),
 units.bytes(units.kibibytes(1)).value(),
 units.kilograms(quectograms$(1)).value(),
 units.bits(quebibytes$(1)).value(),
 units.cubic-meters(units.cube(units.centimeters)(1)).value())
// => 0.9144 exactly, 3.785411784 exactly, 4.54609 exactly,
//    1000, 1024, 1/10^33, 10141204801825835211973625643008, 1/1000000
```

A prefix call accepts an eligible **factory** and returns a new factory. It never prefixes a measurement or mutates its parent. Named catalogue factories such as `kilometers`, `milligrams` and `kibibytes` use this same protocol. Their definitions remain inheritable; measurements remain immutable.

Operation	Result
`si-prefixes.kilo(meters)`	Length factory, scale 1000, symbol `km`
`si-prefixes.milli(grams)`	Mass factory, scale 1/10^6 kg, symbol `mg`
`binary-prefixes.kibi(bytes)`	Information factory, scale 8192 bit, symbol `KiB`
`binary-prefixes.kibi(bytes-per-second)`	Information-rate factory, 8192 bit/s
`square(centimeters)`	Area factory, scale 1/10^4 m², symbol `cm²`
`cube(centimeters)`	Volume factory, scale 1/10^6 m³, symbol `cm³`

The API uses American English names (`meters`, `liters`) and international scientific symbols. Symbol case matters: `mA` and `MA`, `kB` and `KiB`, differ. Symbols are metadata; no automatic parsing of ambiguous symbols is introduced.

Eligibility and powers

Decimal prefixing requires `prefixable: yes`; binary prefixing requires `binary-prefixable: yes`, kind `information` or `information-rate`, and the corresponding dimension vector. Both require positive scale, zero offset, linear arithmetic and an empty `prefix-name`. The returned definition has both eligibility flags set to `no` and records its prefix name and system. Double prefixes, affine points, unfilled families and measurements are refused.

The kilogram already contains a prefix. Prefix `grams`, never `kilograms`: `si-prefixes.milli(grams)` produces milligrams; `si-prefixes.milli(kilograms)` is refused. Temperature **differences** may be prefixed when eligible; absolute temperature factories, logarithmic values and gauge factories are ineligible; see special-unit rules.

A prefix on a length applies **before** its power. Use `square(si-prefixes.centi(meters))` or `square(centimeters)`, and similarly `cube`. The area and volume factories themselves are not prefixable. Thus this API refuses `si-prefixes.centi(square-meters)` instead of silently interpreting it as either 1/100 m² or 1/10000 m². `square` and `cube` accept complete, linear, positive-scale length definitions with the length dimension vector.

Prefix data

A prefix is an immutable instance of `prefix`, with `name`, `symbol`, `radix`, `exponent`, `factor` and `source`. Applying the instance transforms a definition. The factor is an exact integer or rational; it is not a binary64 approximation.

Decimal prefix	Symbol	Exponent of 10
quetta	Q	30
ronna	R	27
yotta	Y	24
zetta	Z	21
exa	E	18
peta	P	15
tera	T	12
giga	G	9
mega	M	6
kilo	k	3
hecto	h	2
deca	da	1
deci	d	-1
centi	c	-2
milli	m	-3
micro	µ	-6
nano	n	-9
pico	p	-12
femto	f	-15
atto	a	-18
zepto	z	-21
yocto	y	-24
ronto	r	-27

Decimal prefix	Symbol	Exponent of 10
quecto	q	-30

Binary prefix	Symbol	Exponent of 2
kibi	Ki	10
mebi	Mi	20
gibi	Gi	30
tebi	Ti	40
pebi	Pi	50
exbi	Ei	60
zebi	Zi	70
yobi	Yi	80
robi	Ri	90
quebi	Qi	100

Local extension and provenance

`prefix(name, symbol, radix, exponent, source)` constructs a local prefix. Names and symbols must be nonempty, radix must be 2 or 10, and the exponent must be an integer. The standard namespaces contain only the listed standard entries; a locally defined prefix is the application's explicit convention. No global registration occurs.

Prefixing inherits the original kind, dimensions, methods, `exact` flag, `reference-edition` and `quantity(number)` construction override. It combines source provenance and multiplies both scale and absolute `scale-uncertainty` by the prefix factor. The parent definition remains unchanged.

`square` and `cube` create area/volume factories and propagate factor exactness, source and edition. For measured length factors they record first-order standard uncertainty as `2*s*u` or `3*s*s*u`. This metadata describes conversion-factor uncertainty; OpenGraf does not provide general measurement uncertainty propagation.

Numeric boundary

Prefix factors, powers of factory scales and coherent conversion multipliers/ divisors use Graf's explicit `exact-product` and `exact-quotient`. Exact rational inputs remain exact even beyond Int64; a `float64` input retains Graf's usual floating-point participation and integer normalization. Symbolic operands remain symbolic. An exact unit factor does not make a user's approximate input exact.

Other arithmetic messages (`plus`, `times`, `over`, etc.) still follow Graf's ordinary numeric promotion rules. For example, a later generic multiplication of a large rational can produce `float64`. This does not affect the exact prefix metadata or the coherent scale conversion path. See angles and Graf's numeric reference.

Migration from historical units

The replacement targets current `Graf/develop`. It changes imports, extension protocols, several names and some semantics. It does not provide automatic backward-compatible aliases. The original implementation is pinned at OpenGraf `1397b974`, just before the unit rewrite. The machine-readable `tools/units-migration.json` accounts for all **31 historical factories and four historical test cases**.

Imports and test execution

Replace `with {open.units, open.units.distance, ...}` and the old repository setup with an explicit import. Paths are relative to the importing file:

```
import(units: graf {path: "../../src/units"});
units.meters(units.kilometers(1)).value()
```

This is the path used by `examples/units/*.graf`. Factories are direct members of `units`; `length`, `mass` and the other families are extension bases, not folders containing their old factories. An import of the entire library is unnecessary.

Run the modern targets from the OpenGraf repository root:

```
graf test tests/units
graf test tests/testing
graf test tests/units-test.graf
```

The retained `tests/units-test.graf` file forwards to `tests/units/legacy-cases.graf`, which expresses its original four cases in gunit. The directory target already includes those cases once. The single-file command is a focused migration check, not another set to add to the directory count.

`setup.graf`, `run-tests.graf` and `run-tests.bat` belong to the old library/test runner. They are not used to validate current units. Other legacy library sections and tests remain outside this migration; scanning all of `tests` is not the supported units command.

Historical test	Modern case in `legacy-cases.graf`
`Distance Conversions`	`verify-distance-conversions`
`Distance Arithmetic`	`verify-distance-arithmetic`
`Temperature Conversions`	`verify-temperature-conversions`
`Comparable`	`verify-closed-interval-comparison`

Use `common.testing.gunit.test>> verify-name(): (...)` rather than `test-fixture` / `test("name")`. `assert.are-equal` becomes `common.testing.gunit.assert-equals`; `assert.is-true` becomes `assert`. Collect expected and actual values in tuples when asserting a required response: an empty stream must fail the assertion instead of skipping it. The testing reference documents deferred failures and discovery.

Family and factory names

Historical family	Current family	Important distinction
`distance`	`length`	Named factories now live directly under `units`.

Historical family	Current family	Important distinction
`weight`	`mass`	Grams, kilograms and avoirdupois pounds measure mass. Force is separate.
`information-unit`	`information`	Coherent coordinate is bits; prefixes state decimal/binary intent.
`temperature`	`temperature-point` / `temperature-difference`	Absolute points and intervals have different arithmetic.
`angle`	`angle`	Radians/degrees are direct factories; conversion retains symbolic pi.
`unit`	`unit`	Measurements now compose a number with an explicit factory and dimensions.

The following table covers every historical factory. `units.` is omitted from modern names. Preserved spellings can still have changed behavior as described below; this is not a promise to reproduce old defects.

Historical module	Historical names	Current replacements
`distance`	`kilometers`, `meters`, `centimeters`	Same names
`distance`	`milimeters`	`millimeters`
`distance`	`miles`, `yards`, `feet`, `inches`, `micrometers`	Same names; international definitions and corrected micro symbol
`angle`	`radians`, `degrees`	Same names; numeric inputs are in the selected unit; degree conversion corrected
`weight`	`kilograms`, `grams`, `pounds`	Same names, in family `mass`
`weight`	`ounces`	`avoirdupois-ounces`; `troy-ounces` is a different explicit convention
`temperature`	`celsius`, `fahrenheit`	Same names, representing points
`temperature`	`kelvin`	`kelvins`, representing points
`information-unit`	`bits`, `bytes`	Same names
`information-unit`	`kilobytes`, `megabytes`, `gigabytes`, `terabytes`	`kibibytes`, `mebibytes`, `gibibytes`, `tebibytes` to retain old binary magnitudes
`information-unit`	`petabytes`, `exabytes`, `zettabytes`, `yottabytes`	`pebibytes`, `exbibytes`, `zebibytes`, `yobibytes` to retain old binary magnitudes
`information-unit`	`brontobytes`, `geopbytes`	`robibytes` (2^90 bytes), `quebibytes` (2^100 bytes)
`information-unit`	`saganbytes`	Explicit local 2^110-byte definition; no standard catalogue alias

The historical byte-prefix factories used 1024 powers under decimal-looking names. Current `kilobytes` means 1000 bytes, whereas `kibibytes` means 1024. Choose which intent to preserve before changing a name. Correct symbols are `kB` / `KiB`, `MB` / `MiB`, and so on. The historical `MG`, `BB`, `OB` and `SB` spellings are not installed as unit aliases. The largest old scale can be reconstructed locally with `prefix("legacy-sagan", "legacy-SB", 2, 110, source)` applied to `bytes`; the source must identify it as a local historical convention. See the prefix reference and its primary sources.

Definitions, extension and payloads

Historical API	Current replacement
`distance(rate, suffix)`	`length.definition(id, symbol, coherentScale)`
`unit(fromCallable, toCallable, suffix)`	A family definition with explicit conversion metadata or overridden conversion methods
`internal-constructor(type, referenceValue)`	Override `factory.quantity(number)` when construction needs validation; return an immutable measurement with `amount` and `factory`
`reference-value`	`.coherent-value()` for calculations; not an automatic stored-data migration
`sufix`	`.symbol` metadata
`.value()`	Same name, now the stored/display numeric payload
`.to(text)`	Explicit Graf string template over `.value()` and `.symbol`
`compared-to`	Same numeric comparison protocol, with kind/dimension checks
`comparable(q).between(a, b)`	`q.gte(a).and(q.lte(b))`; use `gt` / `lt` for open endpoints

The generic linear/affine mapping is `coherent = value × scale + offset`. Exact scale literals use forms such as `0.01(0)` or `1.exact-quotient(100)`. A plain `0.01` is a Graf floating value. Constructing a unit preserves the supplied number's type; it does not silently turn every payload into a rational or float.

Functions now name their results. The provisional names introduced during the rewrite (`make`, `define`, `with-value`, etc.) are separately mapped in naming conventions. They were not the historical unit API. The current core, operation and special references define the extension protocols. A custom conversion must provide both directions consistently and declare kinds, dimensions and linear-arithmetic eligibility. A nonlinear mapping is not made linear by giving it a scale field.

Formatting belongs to the presentation boundary. For example, a float payload can be rendered with `"{0:.2f} {1}"(q.value(), q.symbol)`. This returns a string, not a changed measurement. Formatting support depends on the numeric type and specifier supported by Graf; an unsupported specifier/type is a Graf format failure. Do not force a rational or symbolic payload into a float merely to implement a measurement constructor.

Stored values and coherent references

Historical `reference-value` was relative to each old family's implicit basis. The bases are now explicit and consistent:

Family	Historical reference	Current coherent coordinate
Length	meters	meters

Family	Historical reference	Current coherent coordinate
Mass	grams	kilograms
Information	bytes	bits
Temperature points	degrees Celsius	kelvins
Angles	radians, with a defective degree delegate	radians, with symbolic conversion

When importing stored data, first identify whether it holds a displayed payload or an old reference value, and retain the original unit and convention. Reconstruct a displayed payload through its new factory. For an old reference value, construct the old coherent unit explicitly (`grams`, `bytes`, `celsius`, etc.), then convert to the desired factory. Do not copy that number into a new `.amount` field. The mapping cannot infer the missing unit or repair a value produced by the old degree bug without original input data.

`radians(20)` is twenty radians; `radians(degrees(20))` performs degree conversion. The historical degree delegates ignored their arguments and used a short decimal pi approximation. Those results are not reproduced by the replacement.

Arithmetic and error changes

Mixed plus/minus for compatible readings retains the left factory. Scalar times/over also retains it. Quantity times/over now produces a coherent compound with composed dimensions; a named factory classifies that compound explicitly. Kinds with equal dimensions can still be incompatible (energy/torque, for example). A bare number cannot be added to a measurement. Division by zero remains a Graf numeric error, and unsupported backend operations propagate their Graf failure.

Temperature conversion uses points, with a lower boundary at zero kelvin. Point minus point yields an interval. Point plus/minus interval yields a point. Point plus point adds displayed readings in the left scale: `celsius(20).plus(celsius(10))` returns 30 °C. Scalar times/over also retains the temperature factory. Explicit intervals remain available for temperature changes independent of the scale origin. Logarithmic and gauge readings support the same compatible plus/minus and scalar operations, retaining their convention/reference. Chemical-equivalent and dose-weighted conversions need the explicit references/models documented in special units.

Measurements are immutable instances. Replace an amount through `.quantity-with-value(number)`; do not assign `.amount`, extend a measured instance, or alter a definition's scale after using it.

Validation and coverage

`python3 tools/units_audit.py --legacy-source` verifies that the ledger covers all factories and tests in the pinned historical sources, all 302 published factories have a dedicated catalogue test, every referenced migration case exists, and all seven documentation examples match their executable source. It needs that commit in local git history. Without `--legacy-source`, the same current-tree checks run without accessing historical git objects. This is a correspondence check, not execution or a proof of numerical correctness.

Run `python3 tools/units_catalogue.py --check` to detect drift in generated sources/reference/tests. Execute `graf test tests/units` for numerical, operation, extension and failure checks; independent anchors complement catalogue round trips. The validation matrix identifies the tests for each contract. A complete checkout of `Graf/develop` is separately checked with `dotnet test Graf.Testing/Graf.Testing.csproj`. No unrelated legacy suite is silently included in these totals.

Executable migration example

Run `graf run examples/units/migration.graf`. This example preserves old binary magnitudes where requested and uses current spelling, arithmetic and formatting.

```
// Current replacements for the historical units API; no legacy setup is loaded.
import(units: graf {path: "../../src/units"});

length$ = units.kilometers(1).plus(units.meters(2000));
interval$ = units.celsius(20).minus(units.celsius(10));
furlongs$ = units.length.definition("local-furlong", "fur", 201.168(0));
legacy-large$ = units.prefix("legacy-sagan", "legacy-SB", 2, 110,
    "explicit legacy 2^110 byte convention, not an IEC prefix")(units.bytes);
display$ = units.meters(2.5);
formatted$ = "{0:.2f} {1}"(display$.value(), display$.symbol);
inside$ = units.meters(2000).gte(units.kilometers(1)).and(units.meters(2000).lte(units.kilometers(3)));

(length$.value(),
 units.meters(units.millimeters(1)).value(),
 units.grams(units.avoirdupois-ounces(1)).value(),
 units.bytes(units.kibibytes(1)).value(), units.bytes(units.kilobytes(1)).value(),
 units.bytes(legacy-large$(1)).value(),
 units.degrees(units.radians(pi)).value(),
 units.celsius(units.fahrenheit(32)).value(), interval$.value(),
 units.meters(furlongs$(2)).value(), formatted$, inside$)
// => 3, 0.001, 28.349523125, 1024, 1000, 1298074214633706907132624082305024,
//    180, 0, 10, 402.336, "2.50 m", yes
```

Units: design contract

The implementation includes the unit core, conversion/arithmetic and angles/trigonometry, including local extensions, explicit compound classification, `radians`, and `degrees`, the 290-factory linear/angular catalogue, 87 original families and decimal/binary prefixes. Special-unit rules add 12 named definitions, retained reference metadata, and explicit physical models. This document specifies the implemented unit contract. The historical migration, website and PDF share the same technical documentation sources.

Object model

`unit` is the common base. Quantity families (`angle`, `length`, `area`, `volume`, `mass`, and others) inherit from it. Concrete unit definitions inherit a family; measurements are immutable instances composing a Graf `number` payload with a reference to their defining factory. Numbers are neither subclassed nor mutated. Decoration remains available for optional behavior such as formatting or tracing.

Factories accept a number in their own unit, or convert another compatible quantity. `radians(20)` means twenty radians. `radians(degrees(30))` converts thirty degrees to radians. `.value()` reads the displayed payload. Mixed addition and subtraction preserve the left operand's factory.

Dimensions and semantic kinds

Dimensions contain integer exponents for the seven SI base dimensions and information. Multiplication/division combine dimensions; integer powers multiply them. A root must leave integral exponents. A compound result is explicitly classified by a named factory rather than inferred through a global registry.

Kinds retain distinctions that dimensions alone erase: angle versus ratio, energy versus torque, frequency versus radioactivity, and absorbed versus equivalent radiation dose. Direct conversions between those kinds are refused. An application can classify a force-times-length compound as work or torque.

Numeric and conversion protocol

Linear/affine conversions use `coherent = amount * scale + offset` and the inverse mapping in the target factory. Exact decimal factors use rational numbers; angular factors retain symbolic pi. Arithmetic delegates to the supplied numeric backend, including its supported operations and explicit precision controls.

Angles implement `.sin()`, `.cos()`, `.tan()`, `.cot()`, `.sec()` and `.csc()` with optional precision (default 15), delegating to `common.math` in radians. Concrete factories implement `.asin`, `.acos`, `.atan`, `.acot`, `.asec`, `.acsc` and `.atan2(y, x)`; their results are angular quantities in that factory. Inverse precision controls the radian calculation before display conversion. Logarithms operate on explicit dimensionless ratios, not dimensionful values.

Special quantity rules

- Temperature points and temperature differences are separate kinds. Point minus point produces a difference; point plus/minus difference produces a point. Point plus point adds readings in the left scale and returns a point. Scalar multiplication/division scales displayed readings. Dimensional products of points require explicit differences. Absolute temperatures below zero kelvin are rejected.
- Decimal SI prefixes and binary information prefixes are separate APIs. A prefixed definition cannot be prefixed again. Prefix grams for kilogram units.
- Power and amplitude decibels use explicit linear-ratio conversions. Their physical relationship requires a model. pH uses activity, not automatic molarity.
- Gauge pressure requires an explicit reference pressure. Chemical hardness needs a specified equivalent basis. Gray-to-sievert conversion needs a weighting model.
- Fixed durations are distinct from calendar months and years.

Extensibility

Definitions carry `id`, `symbol`, `kind`, `dimension`, `scale`, `offset`, prefix eligibility, and exactness/source metadata. Measured factors carry `scale-uncertainty` and `reference-edition`; prefix metadata records the applied name/system and prevents repeated prefixing. Measurements add `amount` and `factory`. Users extend families through inheritance or a definition factory such as `length.definition("furlong", "fur", exactFactor)`. New dimensions/kinds compose the same protocol; no mutable global registration is required.

Validation and documentation

Every catalogue unit needs constructor and conversion tests through `common.testing.gunit`. Independent physical anchors complement round trips; round trips alone cannot detect wrong factors. Cover incompatible kinds, compound arithmetic, symbolic angles, temperature boundaries, prefix rules, logarithmic domains, and contextual conversions.

Gunit must invoke every setup and test through Graf's `try`. `TryCatch` converts .NET exceptions to Graf failure results. Catch dispatch receives the exception value with `ErrorLevel = 0`; a matching catch returns its body results and an unmatched failure retains its original positive level. Stored caught exceptions remain inspectable data. Test discovery must traverse only relevant lineage, not method bodies, nested compositions, or imported environments.

Documentation is a technical reference with getting-started instructions, API contracts, conversion conventions, a catalogue, extension guidance, and testing. Markdown sources should support a documentation website and PDF rendering. Gieck's handbook is a coverage reference, not the source of Open Graf's structure or presentation. Check definitions against BIPM, NIST, and CODATA; identify historical conventions, errors, and measured rather than exact constants.

Public API contract

Conversion and arithmetic below are implemented with explicitly defined local factories, as shown in the operation reference. Ready-made `radians`/`degrees` and their trigonometric methods are also implemented. The catalogue reference lists the concrete factories now available. The special-unit reference specifies absolute temperatures, logarithmic and referenced factories, and contextual models.

```
meters(2).plus(centimeters(30)).value()     // 23/10, in meters
centimeters(meters(2)).value()             // 200
radians(degrees(180)).value()              // symbolic pi
radians(2.5).sin()                         // delegates to common.math
radians(2.5).sin(30)                       // explicit numeric precision
kilometers(10).over(hours(2))              // compound velocity
meters-per-second(kilometers(10).over(hours(2)))
```

An unadorned number passed to a factory is always measured in that factory's unit. A bare scalar passed to addition/subtraction is refused; it has no unit. Scalar multiplication/division retain the factory. Division by zero and numeric domain failures use the numeric backend's Graf exceptions. No implicit rounding or conversion to binary floating point is added by the unit layer. Native arithmetic retains Graf's promotion rules, including its existing large-rational precision boundary documented in the angular reference.

Operation	Result contract
`factory(number)`	New immutable quantity in that unit
`factory(quantity)`	Convert compatible kind, or classify a compound with matching dimensions
`quantity.value()`	Original/display payload, still a Graf number

Operation	Result contract
`quantity.coherent-value()`	Payload in the family's coherent unit
`quantity.plus(other)` / `.minus(other)`	Compatible kind, left factory retained
`quantity.times(number)` / `.over(number)`	Left factory retained
`quantity.times(quantity)` / `.over(quantity)`	Coherent compound and composed dimensions
`quantity.pow(integer)`	Compound with multiplied dimension exponents
`quantity.root(integer, precision)`	Compound with divided integral exponents; numeric root delegated to Graf
`factory.quantity(number)`	Overridable numeric construction protocol used by named results
`quantity.rounded(places)`	Explicit rounding of displayed payload
`angle.sin(precision)` / `.cos` / `.tan` / `.cot` / `.sec` / `.csc`	Dimensionless Graf number; default precision 15
`angular-factory.asin(ratio, precision)` and other inverses	Angular quantity in the receiver factory
`family.definition(id, symbol, scale)`	Local inheritable factory; no registration side effects

Factories and quantities share inheritance for method reuse, but a quantity's `factory` reference is explicit. A result must be constructed through that factory's `quantity(number)` protocol rather than by inheriting from an already measured instance. Unit metadata is immutable and overridden explicitly so parent metadata does not appear as extra answers.

Representation and dispatch

A dimension value has fields for length, mass, duration, current, temperature, amount of substance, luminous intensity, and information. Each field is an integer exponent. An angle has zero exponents and a distinct kind. Dimensions compare field by field; semantic kinds compare stable local identifiers.

A definition contains conversion metadata and methods, not a displayed amount. Its numeric constructor creates an instance carrying `amount` and `factory`. Its quantity constructor checks kind and dimensions, converts through coherent values, then invokes the target numeric constructor. A compound has coherent scale one, offset zero, and kind `compound`. Its dimensions are composed from the operands and can be explicitly classified by a named factory.

This uses inheritance for families and factory extension, instantiation for measurements, and composition for numeric payloads and dimensional metadata. Decorating the numeric object would make conversions and arithmetic depend on decoration order. Subclassing every numeric backend would duplicate numeric behavior. Neither is part of the unit core.

Temperature points override addition/subtraction rules and reject compound arithmetic. Logarithmic factories expose explicit ratio conversion rather than pretending that their scale is a linear multiplier. Contextual conversions take a reference or model as an argument and never consult hidden global state.

Prefixes and naming

The catalogue uses unambiguous English factory names and scientific symbols as metadata. Symbols are case sensitive. Do not resolve ambiguous `ton`, `gallon`, `calorie`, or `year` names without naming their convention. Preserve existing Spanish/English documentation independently of code names.

SI prefixes are decimal powers from quetta to quecto. Information uses separate decimal and binary prefix factories. A prefix multiplies the unit's scale with `exact-product`, keeps the family/kind, and adds provenance. Coherent conversion uses the explicit `exact-product` / `exact-quotient` protocol without changing native ordinary promotion. Use `square(prefixed-length)` / `cube(prefixed-length)` for powers. Prefixing an affine point or an already prefixed definition is refused. Area and volume prefixes apply before the power: a square centimeter is `(1/100)^2` square meters.

Catalogue coverage plan

Group	Families and conventions to cover
Geometry	Angle, solid angle, length, area, volume; metric, international foot/inch, nautical mile, liter; U.S. and Imperial volumes explicitly distinguished
Motion	Fixed duration, velocity, acceleration, angular velocity, frequency; calendar periods handled separately
Mechanics	Mass, force, pressure, energy, torque, power, momentum, density, specific volume, dynamic and kinematic viscosity
Thermal	Temperature points/differences, heat capacity, specific heat, thermal conductivity and expansion, heat flux
Electrical	Current, charge, potential, resistance/conductance, capacitance, inductance, magnetic flux/density/field, resistivity and conductivity
Light	Luminous intensity, luminous flux, illuminance and luminance; optical quantities retain their photometric/radiometric distinction
Chemistry	Amount, molar mass, concentration, activity-based pH; hardness names its equivalent basis
Radiation	Radioactivity, absorbed dose, equivalent dose and exposure; model-dependent weighting remains explicit
Information	Bits, bytes, decimal/binary multiples and transfer rate
Ratios	Dimensionless ratios, percentages, power/amplitude decibels with explicit domains

A catalogue entry records definition source, convention, exactness, and any validity restrictions. A measured constant includes its reference edition and uncertainty information. Historical gravitational units name standard gravity rather than using an unspecified local gravitational field.

Sources and conversion policy

The catalogue must distinguish definitions, conventional conversion factors, and measured constants. A printed decimal does not become exact unless its source defines it as exact. Record historical or regional conventions in names and metadata.

Primary references

- BIPM, The International System of Units (SI), ninth edition and published updates: https://www.bipm.org/en/publications/si-brochure.
- NIST, SI Units, links to the defining constants, base and derived units, prefixes, SP 330 and SP 811: https://www.nist.gov/pml/owm/metric-si/si-units.
- NIST, Guide for the Use of the International System of Units (SI), SP 811, Appendix B: https://www.nist.gov/pml/special-publication-811.
- CODATA recommended values, distributed by NIST: https://physics.nist.gov/cuu/Constants/. Record the recommendation's year when using a measured value such as the atomic mass constant.

NIST's SI overview identifies seven base units, 22 derived units with special names and 24 decimal prefixes. Its links and BIPM publications are the starting points for checking current definitions; historical tables need edition checks.

Reviewed angular definitions

The radian and degree factories were checked on 2026-10-24 against BIPM, SI Brochure, ninth edition, version 4.01, Tables 4 and 8: https://www.bipm.org/documents/20126/41483022/SI-Brochure-9-EN.pdf. The radian is the coherent SI unit of plane angle. One degree is exactly (pi/180) radians; the implementation retains symbolic pi rather than a rounded decimal factor. NIST SP 330 (2019) independently confirms that relation: https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.330-2019.pdf.

Prefix eligibility is explicit: radians allow prefixes and degrees do not. The reviewed catalogue uses these same angular definitions.

Secondary coverage reference

Kurt Gieck and Reiner Gieck, Manual de fórmulas técnicas. Alfaomega/Marcombo, 2003, ISBN 970-15-0840-8 (Alfaomega), ISBN 84-267-1330-0 (Marcombo), Spanish copy supplied for this project. Bibliographic data was checked on PDF page 4. The general nomenclature on PDF pages 9-10 and units section A1-A9 guide coverage, not factors or architecture.

Open Graf's architecture and technical documentation are independently organized. The handbook is not a specification of the library, and its wording, tables or chapter organization are not copied into the implementation.

Reviewed catalogue and source identifiers

Reviewed on 2026-10-24. The catalogue records a source ID on each factory; `tools/units-catalogue.json` holds the reviewed data and source map. `tools/units_catalogue.py` deterministically writes the Graf sources, catalogue reference and per-factory integration checks. Run it with `--check` to verify reproducibility. Hand-maintained physical anchors, defining equations and prefix tables are independent of that generation.

Source ID	Use and primary reference
`BIPM-SI-9-v4.01`	SI base/derived definitions and dimensions, Tables 2, 4-6; 24 decimal prefixes, Table 7; accepted/historical non-SI units, Table 8; all 10 binary prefixes, chapter 3. Brochure.

Source ID	Use and primary reference
IEC-80000-13-2025	Binary prefix convention; edition 2 adds the newer prefixes. Official publication. BIPM v4.01 explicitly lists robi/2^90 and quebi/2^100. NIST's older binary summary also defines byte as 8 bit.
NIST-SP811-2008	Appendix B.8: engineering/customary relationships. Rounded display factors are reconstructed from the exact defining conventions, rather than treated as new exact decimals.
international-yard-pound-1959	Yard 0.9144 m; foot 0.3048 m; inch 0.0254 m; avoirdupois pound 0.45359237 kg. NBS 1959 report. NIST foot history distinguishes the retired U.S. survey foot; catalogue feet/acres use the international foot.
standard-gravity-9.80665	Conventional standard gravity 9.80665 m/s², as listed in NIST B.8; force and gravitational engineering units explicitly use this convention, not local gravity.
CODATA-2022-atomic-mass-constant	Dalton conversion nominally 1.66053906892×10^-27 kg, standard uncertainty 5.2×10^-37 kg. NIST SP 961, May 2024, CODATA 2022. exact: no, edition and absolute uncertainty are stored; no uncertainty propagation for measurements is claimed.
fixed-duration-convention	A week is seven fixed 86400-second days; a Julian year is exactly 365.25 such days. These are fixed durations, not calendar arithmetic or a prediction of leap seconds.

Exact derived conventions include 231 in³ per U.S. liquid gallon, 42 such gallons per petroleum barrel, 128 U.S. fluid ounces per U.S. liquid gallon, 4.54609 L per Imperial gallon and 160 Imperial fluid ounces per Imperial gallon. Mass conventions distinguish avoirdupois/troy ounces and short/long tons. The calorie conventions are thermochemical (4.184 J) and International Table (4.1868 J). The International Table Btu is exactly 1055.05585262 J; the exact value is confirmed in NBS gas heating-value guidance, A3.2.2. Mechanical horsepower is derived from 550 ft·lbf/s, metric horsepower from 75 kgf·m/s and electrical horsepower from 746 W. Slug and technical mass unit use their named force/acceleration definitions. CGS conversions are explicitly the conventional NIST SP 811 relationships; no implicit electromagnetic model or measured vacuum constant is substituted.

Coverage and corrections relative to the secondary reference

Reference concern	Catalogue treatment
Space, angle and time	Metric/international geometry, square/cubic factories, angles, fixed durations, angular rates and acceleration.
Mechanics and fluids	Mass/force, energy/torque, pressure, momentum/inertia, density, flow and viscosities retain separate kinds.
Heat	Temperature differences, heat capacity, conductivity, expansion and heat flux. Absolute points and boundaries are implemented in the special-unit API.
Electricity and magnetism	SI named units and coherent electrical/magnetic engineering units, including distinct active/apparent/reactive power kinds.

Reference concern	Catalogue treatment
Light and optics	Photometric/radiometric kinds, luminance versus illuminance, optical power.
Chemistry, radiation and information	Linear chemical/radiation units and decimal/binary information units; reference/model-dependent conversions are explicit in the special-unit API.
Historical SI definitions in A1-A2	Use the modern constant-based SI, not the prototype kilogram or older kelvin/ampere definitions.
Prefix table A3	Expand the older coverage to all 24 current SI prefixes and 10 binary prefixes.
Rounded pound, gravity, mile and horsepower factors in A6-A9	Use exact defining relations; printed rounded equalities do not define the catalogue values.
Imperial gallon, A9 (PDF page 20), item a63	The printed cubic-meter equivalence is dimensionally inconsistent by a factor of 1000. Use 4.54609 L = 0.00454609 m^3, independently checked against NIST.
Pressure and heat conventions	Torr is 101325/760 Pa; no equality with a temperature-dependent mercury column is assumed. Calorie/Btu/horsepower names state the convention.

The catalogue is a finite reviewed set with extensible factories. Derived units are unlimited; the implementation does not claim that every possible unit or every handbook formula is a named catalogue entry. Absolute-temperature, logarithmic, gauge/reference, chemical-hardness and dose-weighting rules are implemented in the special-unit API. Documentation and code are organized independently from the handbook.

Review rules

- Check dimensional consistency before accepting a tabulated factor.
- Verify powers of ten and squared/cubed prefixes independently.
- Distinguish mass from force and energy from torque, even where dimensions or informal naming could conceal the difference.
- Distinguish fixed durations from calendar periods and decay time from decay rate.
- Use current SI definitions for base units; identify obsolete definitions as historical rather than presenting them as current.
- Keep regional liquid measures, calorie conventions, standard gravity and measured physical constants explicit.

Special-unit sources

Reviewed on 25 October 2026. These sources establish definitions; the application still supplies any physical model and its provenance.

Reference	Use in Open Graf
BIPM, SI Brochure, 9th edition v4.01, kelvin/Celsius discussion	Exact Celsius offset and equality of kelvin/Celsius interval magnitudes.
NIST SP 811, Appendix B.8	Fahrenheit/Rankine conversion conventions; point offsets are distinguished from interval factors.

Reference	Use in Open Graf
NIST SP 811, section 8.7	Logarithmic power/field quantities, bel/decibel/neper conventions, explicit references. It supplies no application-specific amplitude-to-power model.
IUPAC Gold Book, pH, 5th edition, 2025	Hydrogen-ion activity definition and molality standard. A concentration-based activity calculation is separately supplied and identified as a model.
NBS Circular 309, Gas-Measuring Instruments	Distinction between absolute pressure and pressure relative to a reference. The API requires the reference explicitly.
USGS, Hardness of Water and Health Canada, Operational Parameters, hardness	Hardness reporting as a chemical equivalent, commonly as calcium carbonate. No unstated chemical basis or rounded atomic mass is installed.
ICRP, Absorbed, Equivalent, and Effective Dose	Absorbed dose and radiation-weighted equivalent dose are different quantities. The caller supplies a weighting model; the unit library does not choose a clinical or protection model.

The hardness examples/tests use explicitly synthetic molar masses to check the equivalent-mass equation independently. They are not physical constants. The radiation and activity examples likewise identify their supplied assumptions. The special-unit API is documented in its technical reference.

Testing with gunit

Open Graf uses `common.testing.gunit` from the updated `Graf/develop` interpreter. The replacement units have 506 targeted tests covering all 302 named factories, the core, conversion/arithmetic, dimensions, local extensions and special rules. The migration guide accounts for all 31 historical factories and the four original unit test cases.

Command

```
graf test tests/testing
graf test tests/testing/gunit-smoke.graf --filter class-filter
```

The auxiliary directory contains 11 cases: 5 execution/assertion cases and 6 discovery cases. The filtered command selects one. Results are plain `PASS`, `FAIL` and `LOAD` lines followed by a summary.

A path is required. A directory scans `.graf` files recursively in ordinal path order, excluding hidden entries, links and other file types. No files outside that target are scanned. Each file loads in its own engine with its source directory; imports are explicit and relative to that file, and are not shared between files. The CLI's existing naming-convention options apply.

`--filter` is a case-sensitive literal substring of `relative/file.graf::test`. A file with several returned values uses IDs such as `cases.graf#2::test`. The filter selects methods before they or their setups execute. File bodies still load; a filter does not avoid loading incompatible source files.

Exit code	Meaning
`0`	At least one selected test, all passed, no load errors
`1`	A test/setup failed, with no load errors
`2`	Invalid target, load/report errors, or no matched tests
`130`	Cancelled with Ctrl+C

A load error takes precedence over failed tests and does not stop the other files from loading. An empty target or filter cannot produce a passing check.

Suite file format

A file must return its suite box, or a stream of suite boxes. Do not call `gunit.test-report` or return report counts in a file intended for `graf test`:

```
suite$ = (mutable>> box {
    count: 0,
    common.testing.gunit.setup>> self$.reset(): self$.count := 0,
    common.testing.gunit.test>> self$.increments(): (
        self$.count := self$.count.plus(1);
        common.testing.gunit.assert-equals(1, self$.count)
    ),
    common.testing.gunit.test>> rejects-missing-label():
        common.testing.gunit.assert-fails {missing}
});
suite$
```

Saving this as `cases.graf` and running `graf test cases.graf` reports two passes and exits zero. Mark setup/test methods with `gunit.setup` / `gunit.test`. They must take no arguments. Declare the receiver in the head when using `self$`. Use receiver fields for mutable state and qualified library paths inside methods. Graf methods do not implicitly capture unrelated caller variables; quoted bodies follow the language's normal binding rules.

Assertions

Here `gunit` abbreviates `common.testing.gunit`:

Assertion	Contract
`assert(condition)`	Condition must be `yes`
`assert-equals(expected, actual)`	Values must compare equal
`assert-not-equals(a, b)`	Values must compare unequal
`fail(message)`	Raise an assertion exception with that message
`assert-fails {expression}`	Evaluate the quoted expression once and require a failure
`assert-fails {exception-class, expression}`	Require a failure matching that class

Braces defer the body. A successful value, including `yes` or an exception returned as data, is not a failure; neither is an empty response stream. A mixed stream satisfies `assert-fails` if at least one failure matches. Successful assertions allow later statements to run.

Execution and reports

Each setup/test is invoked inside Graf `try`. A failed setup skips that test, counts it once as failed, and later tests run their setups again. Several failed responses from one test still count as one failed test. A test with no responses passes. .NET exceptions become Graf exception data through the same mechanism. Setups reset mutable state explicitly; the runner does not clone suites or roll back side effects.

For manual use, `gunit.test-report(suite)` returns a report. `passed` and `failed` count tests; `results.item` rows have `name`, `phase` (`"setup"` / `"test"`) and `outcome`. Failed outcomes contain an inspectable `error`; passed outcomes collect responses in `values.item`. The CLI uses `gunit.filtered-test-report` and reads these reports, without duplicating discovery or the per-method try boundary.

`assert-fails` is a native deferreval that runs its quoted body in the caller's context. Test-local variables and the receiver remain available, and body side effects run once. This prevents unrelated variable errors from making broad negative assertions pass. Every setup/test still runs through Graf `try`.

Discovery

One method is registered per label. Own declarations are inspected newest first, followed by inheritance/instantiation parents breadth-first. Parent ties keep declared order; shared ancestors are visited once. The nearest/latest declaration claims its label before its marker is checked. An unmarked override therefore removes an inherited test/setup; changing a marker replaces that role. Use distinct labels rather than overloading or accumulating tests under one name.

Discovery inspects declarations as data. It does not enter nested data, method bodies or imported environments. Inherited methods use the original suite as receiver. `gunit-discovery-smoke.graf` checks these boundaries.

Compatibility and units target

The other OpenGraf components and tests were written for an older Graf. Only units are being rewritten. Passing gunit checks does not validate those legacy sections. The new unit core has its own targeted suites.

Run `graf test tests/units` for the replacement core. Import the rewritten unit sources explicitly. Do not target the repository root or all of `tests`: unrelated legacy sources are not expected to load or pass on current Graf. `tests/testing` contains contemporary auxiliary gunit checks, not a migration of the legacy library. The retained `tests/units-test.graf` also works with current Graf: it forwards to four migrated cases already included in `tests/units`, so do not add those four to the directory total. The other old test files and the old setup/runner remain outside this target. Value assertions collect their actual response in a tuple, so empty streams fail comparison instead of skipping an assertion. `python3 tools/units_audit.py --legacy-source` checks inventory, migration and example correspondence; it does not replace execution of the tests.

Naming conventions and review

Classes use nouns or adjective+noun phrases that describe what they are. Value-returning methods name their result. A procedure that returns void or no data uses an action verb. An empty result set from a data query does not make the query a procedure. Boolean methods/labels may start with `is`; logical predicates may use Prolog-style relations. Standard gunit `assert*` names are an explicit exception.

This review covers the units implementation, gunit foundation, SDK test command, engine helpers added for those features, their fixtures, examples and technical references. It does not migrate unrelated legacy OpenGraf components or rename all pre-existing Graf APIs. The review was completed before step 7; its naming conventions also apply to the angular API. The current checkpoint is recorded in the eleven-step plan.

Unit API migration

The table below covers provisional names introduced during the rewrite's early steps. The historical migration guide covers the original 31-factory API, including its imports, coherent references and semantic changes.

Previous name	Current name	Result
`make(number)`	`quantity(number)`	Immutable quantity; factory construction extension point
`define(id, symbol, scale, ...)`	`definition(id, symbol, scale, ...)`	Local unit definition
`with-value(number)`	`quantity-with-value(number)`	New quantity through the original factory
`coherent()`	`coherent-value()`	Coherent numeric payload
`to-coherent(number)`	`coherent-value-of(number)`	Numeric payload mapped to coherent units
`from-coherent(number)`	`display-value-of(number)`	Numeric payload mapped to the definition's displayed unit
`can-convert-from(quantity)`	`is-convertible-from(quantity)`	Conversion eligibility

Constructor calls such as `meters$(2)` and `meters$(otherQuantity)` keep their meaning. Overrides of the construction extension point now override `quantity`. This is a naming change; conversion factors, dimensions, numeric behavior, immutability and exception handling are preserved.

Gunit and SDK migration

Previous name	Current name
`gunit.run(suite)`	`gunit.test-report(suite)`
`gunit.run-filtered(suite, prefix, filter)`	`gunit.filtered-test-report(suite, prefix, filter)`
Gunit type `passed` / `failed`	`passed-outcome` / `failed-outcome`
Gunit queries `own` / `marked`	`own-marked-compositions` / `marked-compositions`
Gunit Boolean query `known`	`is-known`
SDK `Graf.TestStaticAsync(...)`	`Graf.TestSummaryAsync(...)`

The report count fields remain `passed` and `failed`. The CLI remains `graf test <PATH> [--filter <TEXT>]`, with the same output and exit codes. Test procedure names now start with verification verbs; suite contents and assertions are unchanged.

In C#, the added failure assertion is named `FailureAssertion`. The unconditional failure deferreval exposed by the gunit work is `AssertionFailureDeferreval`. Their public Graf labels remain `assert-fails` and `fail`. `TestCommand`, `TestSummary`, test classes and quantity-family names already use nouns. New label-result helpers use `LabelResult...` and `LabelValue...` instead of `ResolveLabel...`; SDK helpers name their returned summaries or diagnostics.

Preserved protocols

The mathematical messages `plus`, `minus`, `times`, `over`, `pow`, `root`, `rounded`, `compared-to` and relational comparisons preserve Graf's established numeric protocol and the requested units API. `require-linear` remains a verb: it validates the precondition and returns a statement response. `assert*` names retain testing conventions. Required external overrides such as Spectre's `ExecuteAsync` keep their framework contract names. Deliberately arbitrary fixture labels used in discovery/dispatch tests are test inputs.

No aliases with the retired action names are added. Callers and examples in these repositories use the current names.

Validation and implementation status

Steps 5-9 implement the core, conversion/arithmetic, angles, catalogue, prefixes and special units: 87 original families, 302 named factories, all 24 SI and 10 binary prefixes, immutable measurements and explicit local extension. Absolute temperatures, logarithmic units and reference/model conversions are implemented in step 9. The plan records all eleven steps completed.

Verified checkpoint

Check	Result
`graf test tests/units`	515 passed, 0 failed, 0 load errors; exit 0
`graf test tests/units-test.graf`	4 migrated original cases passed; already included in the directory total
`graf test tests/testing`	11 auxiliary gunit tests passed; exit 0
Complete Graf .NET suite on `develop` (step 11; engine unchanged)	2076 passed, 0 failed, 2 existing printed-form skips
Graf solution build	Successful; symbolic operands and explicit exact-factor arithmetic included
`graf run examples/units/core.graf`	Original 1/3, replacement 2, symbol m, length exponent 1, compatible yes
`graf run examples/units/operations.graf`	23/10, 200, 25/18, 5, 3, yes
`graf run examples/units/angles.graf`	Constructor, symbolic pi/6, 180, mixed sum, sine, exact 1/2, inverse 30 and atan2 135
`graf run examples/units/catalogue.graf`	Exact international foot, U.S./Imperial gallons, decimal/binary byte, quecto gram, quebi byte and cubic centimeter anchors
`python3 tools/units_catalogue.py --check`	Generated sources/reference/integration tests match reviewed data; no stale generated definitions
`graf run examples/units/special.graf`	Temperature, logarithmic/reference, gauge, activity, hardness and radiation anchors
`graf run examples/units/migration.graf`	Historical names, binary magnitude, new definitions/comparison and explicit formatting
`python3 tools/units_audit.py --legacy-source`	302 factory cases, 31 historical factories, 4 original tests and 8 matching documentation examples
Host BigComplex payload check	Identity retained through construction, coherent mapping, conversion and replacement
`mkdocs build --strict`	Successful
`python3 tools/docs_build.py --check`	Published PDF fingerprint/hash and site links/assets/anchors verified
Website browser check (step 11)	All 15 source pages at desktop/mobile widths, PDF download and no JavaScript errors

Check	Result
PDF visual/layout check	All pages rendered with Poppler; no text outside safety margins; catalogue headings repeat

The unit suites cover each family with an independent expected dimension vector, numeric payload kinds, explicit factory references, composite equality, immutable slots, refusal to extend or reopen a measurement, invalid payloads, exact/inexact metadata, inherited overrides, local custom families, compatibility and imports through aliases or the library root. A family copies mutable input dimensions rather than retaining changeable exponent fields.

The 32 operation regressions check independent conversion anchors, round trips, left-factory retention, custom `quantity` validation, scalar and compound arithmetic, all eight exponent operations, exact and precision-controlled roots, explicit classification, numeric failures and incompatible kinds. Actual values are collected in tuples so empty streams fail assertions instead of skipping them.

The 33 angular checks cover independent degree/radian anchors, mixed arithmetic, kind boundaries, metadata overrides, aliases/root imports, six circular functions, seven inverses, principal ranges, axes/quadrants, explicit/default precision, 30-digit rational reference values, periodic inputs, domains, immutability and custom angular factories including overridden construction. Tuples check response cardinality. The 23 new Graf regression cases cover symbolic operands on either side, exact cancellation and all comparison messages. Exact rational reference literals use `a.b(0)` to avoid invoking Graf's large-component promotion when constructing an expected value.

The precision boundary in angles is explicit: `common.math` results retain their backend type, but subsequent generic numeric arithmetic still follows Graf's existing rational-to-float promotion rule.

Step 8 adds 327 checks: one integration test for each of the 290 catalogue factories, 17 independent physical-anchor groups, five defining-equation groups, 12 prefix behavior groups, two explicit reference tables covering every prefix, and a shared-import check. Generated integration cases check construction, factory identity, family/dimension shape, coherent mapping and round trips; hand-maintained anchors/equations catch a wrong factor or vector that a generated round trip alone cannot detect. Prefix tests cover extreme exact factors, byte/bit scaling, powers, kilogram rules, double-prefix refusal, affine and non-information refusal, custom construction, immutable data, source/uncertainty metadata and all 34 standard factors. The native exact-factor API has 11 additional regression cases.

`tests/units` is the replacement test target. The other legacy tests and other library sections still target the obsolete API; they are not evidence for or against the modern unit core. Importing the library root in an import regression reads only the unit labels it uses.

Step 9 validation

The 52 additional checks cover 12 special-factory construction/immutability cases, nine temperature groups, 12 logarithmic groups and 19 contextual/model groups. They check fixed temperature anchors and boundaries, left difference factories, real domains, nonexact log precision/tolerance, all six power/amplitude conventions, pH activity outside 0-14, reference retention and changes, gauge pressure, explicit activity standards, basis-preserving hardness and custom radiation models. Negative tests reject missing models, wrong kinds/dimensions, conflicting references, nonpositive ratios/standards/weights and mutation. Assertions check response cardinality. Current sequence deferrevals are exercised on unit results.

Graf was synchronized to `develop` commit `7da35b5fba689dee011ea5e41667d8b6dbd14359` before implementation. Its full suite has **2076 passed, zero failed and two existing omissions**, including the newly added deferrevals and expression-map cases. Step 9 requires no engine change. The host BigComplex check listed above is the retained step 8 result, not a new host-backend test in this step.

Step 10 validation and contract coverage

The four original unit cases now run through gunit in `legacy-cases.graf`. `tests/units-test.graf` forwards to that suite, preserving the file entry point. Thirteen migration groups check all historical factories, binary-name decisions, corrected degree behavior, implicit reference changes and stored-data reconstruction, explicit definitions/formatting, arithmetic boundaries and deliberately retired labels. These 17 cases bring the modern directory suite to **506 passed**.

`tools/units_audit.py --legacy-source` checks its ledger against historical git sources at `1397b974816c00fe8d535db8f566d90523d412bf`: **31 factories and four original tests** are accounted for. Each of the **302 current factories** has one dedicated catalogue construction test. Six documented examples match their executable files. The audit checks correspondence, not executed code coverage.

Contract	**Executable suites under** `tests/units`
Every published factory	`catalogue/factories-01.graf` through `12`, plus `catalogue/special-factories.graf`
Independent factors and dimensional equations	`catalogue-anchors.graf`, `catalogue-identities.graf`
Number payloads, construction, dimensions, kinds and immutable metadata	`core.graf`, `definitions.graf`, `families.graf`, `imports.graf`
Conversion and left-factory arithmetic	`conversion.graf`, `arithmetic.graf`
Reading arithmetic across all 302 named factories and retained gauge/level references	`reading-arithmetic.graf`
Products/quotients, powers and roots	`compounds.graf`, `powers.graf`
Local inheritance and overridden construction	`extensions.graf`, plus prefix/angle/special extension cases
Angular conversion, forward/inverse math and precision	`angles.graf`, `trigonometry.graf`, `inverse-trigonometry.graf`
All SI/binary prefix factors, eligibility and dimension powers	`prefix-tables.graf`, `prefixes.graf`
Point/interval rules and boundaries	`temperature-points.graf`
Logarithmic precision, domains and local conventions	`logarithmic.graf`
Retained references and explicit physical models	`contextual.graf`
Original cases and historical API/data decisions	`legacy-cases.graf`, `migration.graf`

Negative checks are kept with each contract: invalid types/definitions, wrong kinds or dimensions, zero division, invalid precision/root degrees, immutable slots, unsupported dimensional products of points/levels, nonpositive logarithmic inputs, reference conflicts and missing or invalid models. This is contract coverage; it is not a claim to execute every operator on every possible numeric backend or physical model. Core operations are shared, and numeric backend restrictions remain Graf's responsibility. Step 10 requires no engine behavior change.

Reading arithmetic review

Nine new groups bring the directory suite from 506 to **515 cases**. One group executes addition, subtraction, scalar multiplication/division and factory retention for every one of the **302 named factories**, including all logarithmic and temperature definitions. Five groups cover same-scale and mixed-scale temperatures, exact decimals, negative readings, absolute-zero results, custom affine factories and increments in either operand

position. One group covers mixed logarithmic conventions, pH, exact same-factory readings and incompatible kinds. Two groups cover gauge and logarithmic reference changes during arithmetic, retained references, immutable operands and invalid results.

Existing offset, temperature, logarithmic and contextual cases now exercise reading arithmetic instead of rejecting it. Dimensional products retain their linear-unit requirements; invalid domains, missing models and incompatible kinds remain covered. The eighth executable documentation example is `examples/units/reading-arithmetic.graf` in the operation reference. This review changes the Open Graf library without changing Graf's engine.

Step 11 documentation publication

The website/PDF build reference describes the pinned dependencies, commands and outputs. All **15 Markdown pages** in the navigation are included in the technical PDF; there is one shared source set. The catalogue uses landscape pages and repeated table headers; the rest uses portrait pages. Linked contents, section destinations and outlines support navigation. All 302 factory names are checked in extracted PDF text. Font and source-link rendering, table continuations, code, section transitions and margins were also reviewed visually.

The generated site was opened at desktop and narrow widths, with external requests blocked. All 15 documentation pages were checked for a title and body overflow, and the PDF download was compared with its published hash. The local HTML target/anchor checker also includes the generated 404 page. These checks verify the static build, not a deployment to an external hosting service.

The seventh executable example is `examples/units/quickstart.graf`. The build manifest records the source fingerprint, PDF page count, file hash and navigation counts for this exact publication. The host BigComplex observation is retained from step 8; it is not a newly run host-backend test in this documentation step. The step 11 Graf suite, 506 unit cases and 11 auxiliary gunit cases were rerun without changing the engine.

Graf prerequisites

Use `T-and-Q/Graf` on `develop`. The completed testing foundation ensures:

1. Each gunit setup/test is invoked inside Graf `try`.
2. .NET exceptions pass through `TryCatch`, then catch dispatch receives a level-zero exception value.
3. Matching handlers return their responses; unmatched failures retain their level.
4. Reading a stored caught exception does not raise it again.
5. Handler-body failures propagate independently of dispatch misses.
6. Discovery selects nearest/latest declarations and skips nested data, method bodies and imported environments.
7. `graf test` supports explicit targets, filters, diagnostics and exit codes.
8. `assert-fails` evaluates quoted code in the caller context, including local variables and receiver; side effects execute once. A successful local expression cannot pass by losing its binding and raising an unrelated error.

Graf has 13 gunit execution/assertion regressions, 13 discovery regressions and 9 command regressions. OpenGraf's auxiliary suites add 11 Graf-language checks. The CLI's pass/fail/load/no-match/cancellation codes were verified in step 4.

Contract scope and limits

Concern	Implemented now	Still required
Construction	Core and special constructors, payload/domain validation, retained references and immutable values	Further application-specific factories

Concern	Implemented now	Still required
Dimensions and kinds	Eight integral fields; 87 family vectors; products, quotients, powers and integral roots	Application-specific extensions
Conversion	Linear, affine, logarithmic and reference mappings; independent model anchors; kind/dimension checks	Further application models
Arithmetic	Linear operations, special point/difference rules, reference-aware comparisons and rounding	Further application-specific rules
Angles	Radians/degrees, symbolic conversion, circular/inverse methods, precision, domains and extension	Further catalogue angles; native large-rational promotion decision
Temperature	Four point scales, four difference scales, exact offsets, absolute-zero boundaries and point/difference operations	Population-inversion models are outside the ordinary point API
Prefixes	All SI/binary factors, powers, double-prefix refusal and special-unit exclusions	Application-specific eligibility
Special cases	Logarithmic domains, retained references, gauge pressure, pH activity, equivalent-basis hardness and explicit radiation weighting	Full speciation, uncertainty propagation and radiation-field aggregation are outside these APIs
Extensibility	Local factories, overridden construction, custom ratio/activity/radiation models and explicit equivalent bases	More application-specific models
Migration	All 31 historical factories mapped, 4 original cases migrated, coherent-coordinate changes and semantic decisions documented	Unrelated legacy components remain outside scope
Documentation	Technical Markdown, complete units migration, 302-factory inventory, special/prefix references, reviewed sources and site build	Rebuild and visually review after future source, font or renderer changes

Round trips alone cannot detect a shared wrong factor. The catalogue tests include independent anchors such as the exact international inch, liter and separately named U.S./Imperial gallons. Measured constants require documented reference editions, uncertainty information and tolerance-based tests.

Unit source migration

Legacy file	Replacement
`unit.graf`	Public entry point to the new core class
`angle.graf`	Shared angular family with circular/inverse methods; public `radians.graf` and `degrees.graf` definitions

Legacy file	Replacement
`distance.graf`	`length.graf`
`weight.graf`	`mass.graf`; force is a distinct family
`information-unit.graf`	`information.graf`
`temperature.graf`	`temperature-point.graf` and `temperature-difference.graf`

These names deliberately replace the obsolete unit API. The current checkpoint replaces the original unit catalogue through the explicit mappings in the migration guide. Other old library components are not migrated.

Website and PDF builds

The website and downloadable PDF use the Markdown pages in `mkdocs.yml`, in its navigation order. The PDF is a technical reference with linked sections, code, tables, source references and page numbers. The catalogue uses landscape pages; the API and guides use portrait pages. The overview's website link menu is represented by the PDF's linked contents. There is no separate book manuscript.

Install documentation dependencies

From the OpenGraf repository root, with Python 3.10 or newer:

```
python -m pip install -r docs/requirements.txt
```

PDF rendering also needs Pango/Harfbuzz and DejaVu Sans/DejaVu Sans Mono fonts. On Ubuntu/Debian, install the native dependencies if they are absent:

```
sudo apt-get install libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz-subset0 fonts-dejavu-core
```

The WeasyPrint installation reference has platform-specific instructions. Renderer and Markdown versions are pinned. Native library or font updates can alter pagination; repeat the visual checks after changing them. None of these dependencies is needed to run the Graf units.

Build both formats

```
python3 tools/docs_build.py
```

The command first checks generated catalogue files and unit/example correspondence. It then creates the PDF, copies it into the documentation's download directory, builds MkDocs in strict mode and checks local links/assets and HTML anchors. Any renderer warning or unresolved internal PDF link fails the build.

Output	Purpose
`output/pdf/open-graf-units.pdf`	Standalone PDF artifact
`output/docs/reference.html`	Intermediate combined HTML for diagnosis
`docs/downloads/open-graf-units.pdf`	Published PDF, copied into the generated site
`docs/downloads/build.json`	Documentation input fingerprint, PDF hash and page/ navigation counts
`site/`	Generated static website, including the PDF download

The PDF and manifest in `docs/downloads` are committed together. The other generated outputs are ignored. The fingerprint covers all included Markdown, the navigation, render styles, build/check tools and dependency declarations. It deliberately excludes the generated PDF and manifest, avoiding a circular hash dependency. The reference date is explicit in `mkdocs.yml`. The build restarts its own process with `SOURCE_DATE_EPOCH` set to that date, fixing embedded font timestamps without changing the caller's environment. Repeated builds with the verified renderer, fonts and native libraries have the same PDF bytes; changed fonts/libraries still require a visual review.

Check an existing publication

```
python3 tools/docs_build.py --check
```

This checks the committed PDF's input fingerprint and file hash, then rebuilds and checks the site. It does not silently regenerate an out-of-date PDF. Use the normal build after editing documentation, styles or the build pipeline. To build just the website for local editing, `mkdocs build --strict` also works; the combined check remains the publication check.

Use `mkdocs serve` to preview the website locally. Deploy the contents of `site/` to a static host when a hosting destination is selected. Building a site does not configure GitHub Pages or another hosting service.

Publication checks

The complete checkpoint is recorded in validation. Before publishing updated reference files:

1. Run `graf test tests/units` and the auxiliary `graf test tests/testing`.
2. Run the six detailed examples plus the quickstart.
3. Run the catalogue and migration audits, using `--legacy-source` when the pinned historical commit is present locally.
4. Build both formats and run `tools/docs_build.py --check`.
5. Render PDF pages with Poppler (`pdftoppm`) and inspect code, table continuations, Unicode symbols, page headers/footers and long source links.
6. Open the website at desktop and narrow widths; check navigation, code/table scrolling, source links and the PDF download.

The automated PDF check verifies factory names, text safety margins, section destinations and outlines. It does not replace visual inspection. Local link checks do not claim that every external bibliography URL will remain available. The numerical tests validate units independently of the documentation renderer.

Plan de trabajo y avance

Actualizado: 25 de octubre de 2026.

Los 11 pasos están completados. Units y su documentación técnica están validados; el sitio estático y el PDF se generan desde las mismas fuentes. Units dispone de 302 fábricas con nombre, los 24 prefijos SI y 10 binarios, aritmética dimensional, trigonometría, temperaturas y conversiones especiales. Los contextos y modelos físicos se suministran explícitamente.

Paso	Trabajo	Estado
1	Actualizar ambos repositorios, usando `Graf/develop` y `OpenGraf/master`.	Completado
2	Completar gunit: setups y tests mediante `try`, registro de fallos, continuación y `assert-fails` diferido con filtro de clase.	Completado
3	Revisar el descubrimiento: herencia pertinente, sobrescrituras y exclusión de objetos anidados, cuerpos de métodos y entornos importados.	Completado
4	Implementar `graf test`: archivos, carpetas, filtros, diagnósticos y códigos de salida.	Completado
5	Implementar `unit`, sus familias y las medidas como instancias inmutables con número, fábrica y dimensiones.	Completado
6	Completar conversiones, aritmética mixta, dimensiones compuestas, potencias, comparación y extensibilidad local.	Completado
7	Integrar `common.math`: trigonometría, conversiones con π y precisión explícita, conservando los tipos numéricos de Graf.	Completado
8	Construir el catálogo y los prefijos decimales/binarios; verificar factores con BIPM, NIST y CODATA, usando Gieck como referencia de cobertura.	Completado
9	Resolver temperaturas absolutas y diferencias, unidades logarítmicas y conversiones con referencias o modelos explícitos.	Completado
10	Migrar la API antigua, actualizar ejemplos y probar cada unidad, operación, extensión y error con gunit y la suite completa de Graf.	Completado
11	Completar documentación técnica, catálogo, fuentes y migración; verificar sitio web y PDF y publicar los cambios de ambos repositorios.	Completado: fuentes, sitio estático y PDF verificados; cambios publicados

Último checkpoint verificado

- Graf `develop`: **2076 pruebas correctas, 0 fallos y 2 omisiones existentes**. Se incorporaron antes del paso 9 los deferrevals nuevos hasta `7da35b5fba689dee011ea5e41667d8b6dbd14359`, incluidas sus pruebas de `expression.map`.
- `graf test tests/units`: **515 correctas, 0 fallidas y 0 errores de carga**. Son las 506 del paso 11 más 9 grupos de revisión de aritmética de lecturas.
- `graf test tests/units-test.graf`: **4 correctas**; ya incluidas en las 515.
- `graf test tests/testing`: **11 correctas, 0 fallidas y 0 errores de carga**.
- `python3 tools/units_catalogue.py --check`: **655 archivos generados verificados**.
- Documentación: `mkdocs build --strict` y ocho ejemplos ejecutables verificados (incluido el quickstart). La documentación sigue siendo una referencia técnica, con Markdown compartido para sitio y PDF. El PDF descargable incluye índice, destinos internos, referencias y tablas con encabezados repetidos.

El catálogo lineal conserva sus 290 entradas. Las 12 definiciones especiales completan el inventario de 302. `catalogue.definition` enumera ambas partes; los contextos locales no se registran.

Revisión posterior: aritmética de lecturas

- `celsius(20).plus(celsius(10))` devuelve 30 °C. Las escalas mixtas se convierten a la unidad izquierda antes de sumar. También se permite escalar una lectura térmica con un número y sumar incremento más temperatura.
- La misma regla de suma/resta y escalado se aplica a decibelios, pH, presión manométrica y definiciones locales con origen desplazado. Los resultados conservan su fábrica y referencias; cada fábrica valida los nuevos valores.
- Una prueba recorre las 302 fábricas del catálogo. Ocho grupos adicionales cubren escalas mixtas, límites, extensiones y referencias. Las diferencias térmicas y los modelos físicos siguen disponibles explícitamente.
- La documentación compartida, el ejemplo ejecutable de lecturas y el PDF reflejan estas reglas.

Paso 9 completado

- Cuatro escalas de puntos térmicos y sus diferencias: límites en cero kelvin, conversiones exactas, punto menos punto y punto más/menos diferencia.
- Bels, decibels y nepers con convenciones de potencia/amplitud separadas; pH a partir de actividad. Precisión explícita mediante `common.math`.
- Niveles con referencia retenida en cada medida: cambio de referencia, comparación y recuperación de la magnitud lineal.
- Presión manométrica con referencia explícita; modelos de relación amplitud/potencia, actividad, base química equivalente y ponderación de dosis.
- Factores, dominios y supuestos documentados con fuentes primarias. Las pruebas incluyen anclas independientes, errores, inmutabilidad y extensiones.

Graf no requirió modificaciones del motor para implementar este paso. Se conservan sus reglas de promoción numérica y el uso de `try` en gunit.

Paso 10 completado

Se migran las cuatro pruebas originales a gunit y se conserva su archivo de entrada. Las 31 fábricas históricas tienen reemplazo o decisión explícita: correcciones de nombres, prefijos binarios, puntos térmicos, grados y referencias coherentes. La guía de migración cubre también datos almacenados, extensiones, formato y comparación sin dependencias de la API vieja.

`tools/units_audit.py --legacy-source` verifica el inventario histórico contra el commit conservado, 302 casos de fábrica y seis ejemplos sincronizados con sus referencias. La matriz de validación enlaza las suites de unidades, operaciones, extensiones y errores. Graf no requirió cambios propios.

Paso 11 completado

La referencia técnica reúne API, catálogo, prefijos, reglas especiales, migración, fuentes y validación. Se revisaron las declaraciones de estado y se añadió un quickstart ejecutable. La navegación de MkDocs determina también el orden del PDF; no hay un manuscrito separado ni una adaptación del libro de Gieck.

`tools/docs_build.py` produce los dos formatos. `--check` detecta PDF desactualizado, hash incorrecto, enlaces/recursos locales ausentes y anclas HTML rotas. La generación comprueba los 302 nombres de fábricas en el PDF, sus destinos internos y los márgenes de texto. El sitio se verificó en escritorio y móvil; el PDF se renderizó con Poppler para revisión visual. El manifiesto registra la huella de las fuentes y las medidas del artefacto publicado.

El sitio queda listo para desplegar en un alojamiento estático; este trabajo no configura un servicio de hosting. Graf no requirió cambios propios en los pasos 9-11; su rama `develop` y la publicación de OpenGraf están verificadas.

Alcance y decisiones vigentes

OpenGraf contiene otras secciones de una versión antigua de Graf. Se reescribe **solo units**: el objetivo de prueba es `tests/units`, más `tests/testing` para gunit. Las secciones y pruebas antiguas restantes no se consideran migradas.

Las familias y definiciones usan herencia; las medidas son instancias inmutables que componen un número con su fábrica y dimensiones. La extensibilidad es local, sin registro global mutable. Las reglas de nombres se mantienen en convenciones y migración.

El libro de Gieck sirve como referencia de cobertura. Las definiciones se contrastan con BIPM, NIST, CODATA y las fuentes específicas de cada dominio; las correcciones y la bibliografía están en fuentes. La promoción ordinaria de racionales grandes de Graf sigue siendo una cuestión del lenguaje; las conversiones de escala usan su protocolo exacto explícito.